



مقدمة

الحمد لله الذي علّم بالقلم، علّم الإنسان ما لم يعلم، والصلاة والسلام على من بُعث مُعلماً للناس وهادياً وبشيراً، وداعياً إلى الله بإذنه وسراجاً منيراً؛ فأخرج الناس من ظلمات الجهل والغباء، إلى نور العلم والهداية، نبينا ومعلمنا وقُدوتنا الأول محمد بن عبد الله وعلى آله وصحبه أجمعين، أما بعد:

تسعى المؤسسة العامة للتدريب التقني والمهني لتأهيل الكوادر الوطنية المدربة القادرة على شغل الوظائف التقنية والفنية والمهنية المتوفرة في سوق العمل السعودي، ويأتي هذا الاهتمام نتيجة للتوجهات السديدة من لدن قادة هذا الوطن التي تصب في مجملها نحو إيجاد وطن متكامل يعتمد ذاتياً على الله ثم على موارده وعلى قوة شبابه المسلح بالعلم والإيمان من أجل الاستمرار قدماً في دفع عجلة التقدم التنموي، لتصل بعون الله تعالى لمصاف الدول المتقدمة صناعياً.

وقد خطت الإدارة العامة للمناهج خطوة إيجابية تتفق مع التجارب الدولية المتقدمة في بناء البرامج التدريبية، وفق أساليب علمية حديثة تحاكي متطلبات سوق العمل بكافة تخصصاته لتبلي تلك المتطلبات، وقد تمثلت هذه الخطوة في مشروع إعداد المعايير المهنية الوطنية ومن بعده مشروع المؤهلات المهنية الوطنية، والذي يمثل كل منهما في زمنه، الركيزة الأساسية في بناء البرامج التدريبية، إذ تعتمد المعايير وكذلك المؤهلات لاحقاً في بنائها على تشكيل لجان تخصصية تمثل سوق العمل والمؤسسة العامة للتدريب التقني والمهني بحيث تتوافق الرؤية العلمية مع الواقع العملي الذي تفرضه متطلبات سوق العمل، لتخرج هذه اللجان في النهاية بنظرة متكاملة لبرنامج تدريبي أكثر التصاقاً بسوق العمل، وأكثر واقعية في تحقيق متطلباته الأساسية.

وتتناول هذه الحقيبة التدريبية "لغة برمجة ١" لمتدربي تخصص "الحاسب الآلي" في المعاهد الصناعية الثانوية ومعاهد العمارة والتشييد، موضوعات حيوية تتناول كيفية اكتساب المهارات اللازمة لهذا البرنامج لتكون مهاراتها رافداً لهم في حياتهم العملية بعد تخرجهم من هذا البرنامج. والإدارة العامة للمناهج وهي تضع بين يديك هذه الحقيبة التدريبية تأمل من الله عز وجل أن تسهم بشكل مباشر في تأصيل المهارات الضرورية اللازمة، بأسلوب مبسط خالٍ من التعقيد.

والله نسأل أن يوفق القائمين على إعدادها والمستفيدين منها لما يحبه ويرضاه؛ أنه سميع مجيب الدعاء.

الإدارة العامة للمناهج



الفهرس

رقم الصفحة	الموضوع
٢	مقدمة
٣	الفهرس
٥	تمهيد
٨	الوحدة الأولى: مقدمة في البرمجة ولغاتها
٩	طريقة عمل جهاز الحاسب الآلي
١٠	لغات البرمجة
١١	الفرق بين المترجم The compiler والمفسر The Interpreter
١٢	الخوارزميات
١٤	خرائط التدفق
١٦	حل المشكلات
١٦	جمل التحكم بتنفيذ البرامج
١٩	تمارين الوحدة
٢٣	الوحدة الثانية: مقدمة إلى لغة بايثون
٢٤	لغة بايثون
٢٥	تثبيت لغة بايثون على الجهاز
٢٧	التعرف على بيئة محرر كتابة بايثون Python IDE
٢٨	أساسيات في لغة بايثون
٣٠	دالة print()
٣٢	مهارات في دالة print()
٣٧	كتابة التعليقات في لغة بايثون
٣٩	تمارين الوحدة
٤٤	الوحدة الثالثة: أنواع البيانات والمتغيرات
٤٥	البيانات في لغة بايثون
٤٨	المتغيرات في لغة بايثون Variables
٥٦	تخزين النصوص في لغة بايثون



رقم الصفحة	الموضوع
٦٠	دوال تتعامل مع النصوص في لغة بايثون
٦٥	تحويل نوع المدخلات في لغة بايثون
٦٨	تمارين الوحدة
٧٢	الوحدة الرابعة: العمليات الحسابية
٧٣	المعاملات الحسابية
٨١	أولويات المعاملات الحسابية
٨٢	كتابة التعبيرات الحسابية
٨٢	دوال رياضية جاهزة في لغة بايثون
٨٩	المكتبات في لغة بايثون Modules
٩٢	معاملات الإسناد الرياضية
٩٤	تمارين الوحدة
٩٨	الوحدة الخامسة: العمليات العلاقية والمنطقية
٩٩	معاملات المقارنة
١٠٢	معاملات المساواة
١٠٤	المعاملات المنطقية في لغة بايثون
١٠٦	كتابة التعبيرات الشرطية والمنطقية
١٠٧	توليد الأرقام العشوائية
١٠٩	الأخطاء في البرمجة
١١٢	تمارين الوحدة
١١٦	الوحدة السادسة: الاختيار بالجمل الشرطية
١١٧	الجمل الشرطية في لغة بايثون
١٢٧	الأخطاء الشائعة عند كتابة الشروط في لغة بايثون
١٢٧	المعاملات المنطقية التي يمكن استخدامها في الشروط
١٣٠	تمارين الوحدة
١٣٣	المراجع



تمهيد

الهدف العام من الحقيبة :

تهدف هذه الحقيبة إلى إكساب المتدرب المعارف والمهارات الأساسية في برمجة تطبيقات سطح المكتب باستخدام لغة بايثون.

تعريف بالحقيبة :

تقدم هذه الحقيبة مجموعة من المهارات والمعارف الأساسية لمفهوم البرمجة وأنواع لغاتها المختلفة وخطوات حل المسائل ومن ثم برمجة تطبيقات سطح المكتب باستخدام إحدى لغات البرمجة المشهورة (لغة بايثون Python) ويشمل ذلك أساسيات البرمجة، ومعرفة أنواع البيانات، وتعريف المتغيرات والتعامل مع العمليات (الحسابية والعلاقية والمنطقية) وجمل الاختيار الشرطية وتشمل: (جملة if البسيطة وجملة if-else والجملة الشرطية if-elif-else).

الوقت المتوقع لإتمام التدريب على مهارات هذه الحقيبة التدريبية :

يتم التدريب على مهارات هذه الحقيبة في ٨٠ ساعة تدريبية، موزعة كالتالي:

١٠ ساعات تدريبية	مقدمة في البرمجة ولغاتها	الوحدة الأولى :
١٠ ساعات تدريبية	مقدمة إلى لغة بايثون	الوحدة الثانية :
١٥ ساعة تدريبية	أنواع البيانات والمتغيرات	الوحدة الثالثة :
١٥ ساعة تدريبية	العمليات الحسابية	الوحدة الرابعة :
١٥ ساعة تدريبية	العمليات العلاقية والمنطقية	الوحدة الخامسة :
١٥ ساعة تدريبية	الاختيار بالجمل الشرطية	الوحدة السادسة :



الأهداف التفصيلية للحقيبة:

- من المتوقع في نهاية هذه الحقيبة التدريبية أن يكون المتدرب قادراً وبكفاءة على أن:
١. يمثل الخوارزميات بخرائط التدفق في حل المسائل.
 ٢. يمثل المتغيرات في لغة بايثون.
 ٣. يستخدم المعاملات الحسابية في لغة بايثون.
 ٤. يستخدم الدوال الجاهزة في لغة بايثون.
 ٥. يكتب التعابير الشرطية والمنطقية في لغة بايثون.
 ٦. يستخدم الجمل الشرطية if، if-else، if-elif-else في لغة بايثون.
 ٧. يرسم المخططات الانسيابية للجمل الشرطية.
 ٨. يكتب برنامج بلغة بايثون.



الوحدة الأولى

مقدمة في البرمجة ولغاتها



الوحدة الأولى

مقدمة في البرمجة ولغاتها

الهدف العام للوحدة:

تهدف هذه الوحدة إلى اكساب المتدرب المعرفة بلغات البرمجة واستخدام الخوارزميات ورسم خرائط التدفق.

الأهداف التفصيلية:

- من المتوقع في نهاية هذه الوحدة التدريبية أن يكون المتدرب قادراً وبكفاءة على أن:
١. يحل المسائل باستخدام الخوارزميات.
 ٢. يمثل الخوارزميات بخرائط التدفق في المسائل.
 ٣. يتحكم في سير تنفيذ البرنامج.

الوقت المتوقع للتدريب على هذه الوحدة: ١٠ ساعات تدريبية.

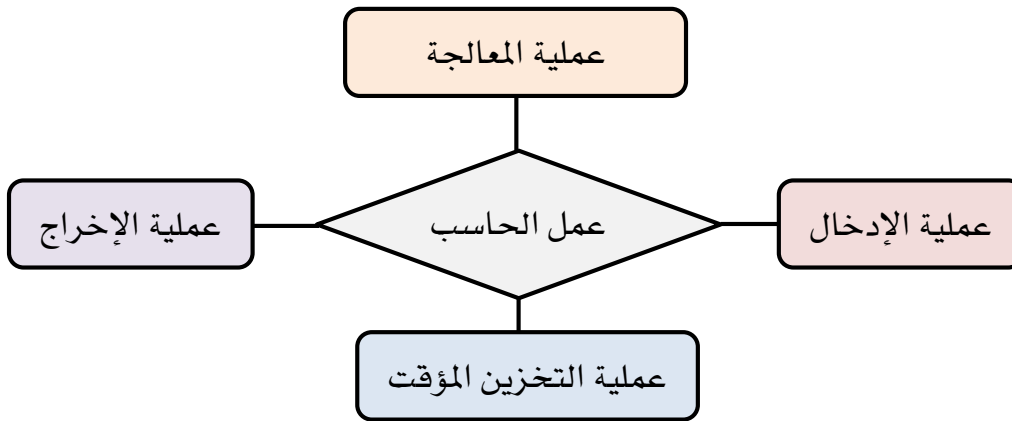
الوسائل المساعدة:

١. جهاز حاسب آلي.
٢. جهاز عرض (Data Show).



طريقة عمل جهاز الحاسب الآلي:

يمر جهاز الحاسب الآلي بأربع عمليات رئيسية أثناء عمله وهي: عملية الإدخال والمعالجة والتخزين المؤقت والإخراج. كما في الشكل التالي تسلسل خطوات عمل الحاسب الآلي:



الشكل (١ - ١)

١. عملية الإدخال:

هي عملية إدخال البيانات من قبل مستخدم جهاز الحاسب الآلي بواسطة وحدات الإدخال المختلفة مثل: (لوحة المفاتيح، الكاميرا، الفارة، الميكروفون).

٢. عملية المعالجة:

هي عملية معالجة البيانات المدخلة من قبل المستخدم إلى معلومات حسب التسلسل والتعليمات المبرمجة مسبقاً وتتم هذه العملية في وحدة المعالجة المركزية CPU.

٣. عملية التخزين المؤقت:

هي عملية تخزين البيانات مؤقتاً بعد معالجتها ثم إرسالها لعملية الإخراج، حيث يتم تخزين البيانات والمعلومات في الذاكرة بلغة الآلة (0 و1)؛ ليتمكن الجهاز من التعامل معها خلال عمل الكمبيوتر.

٤. عملية الإخراج:

هي العملية الأخيرة في سلسلة عمل الجهاز وهي عبارة عن عرض البيانات المعالجة من خلال وحدات الإخراج المتوفرة مثل: الطابعة، الشاشة، السماعة.



تعريف البرنامج:

البرنامج عبارة عن مجموعة أو سلسلة من التعليمات والأوامر التي يحددها المبرمج بواسطة لغة برمجة معينة لتنفيذ مهمة أو حل مشكلة واستخراج النتائج في إطار زمني محدد.

تعليمات أساسية لابد أن توجد في أي برنامج:

المدخلات: هي البيانات والمعلومات التي تدخل بواسطة مستخدم البرنامج.
المخرجات: هي المعلومات التي تظهر على الشاشة أو أي وسيلة إخراج بعد معالجتها من قبل البرنامج.

الحساب: القيام بالعمليات الحسابية والرياضية.
التنفيذ المشروط: تنفيذ العمليات عند تحقق الشروط الموضوعة.
التكرار: تنفيذ العمليات مراراً لحين تحقق الشروط أو حسب المطلوب.

لغات البرمجة:

هي اللغات التي يتم كتابة البرامج من خلالها ، وتنقسم لغات البرمجة إلى ثلاثة أقسام رئيسية هي:

١. لغة الآلة (Machine language):

هي اللغة الوحيدة التي يفهمها الحاسب ويستطيع التعامل معها. حيث تتكون من رقمي 0 و 1 ، وتعتبر الوسيط بين اللغات عالية المستوى والأجهزة.

٢. لغة التجميع (Assembly language):

هي لغة تستخدم اختصارات من اللغة الإنجليزية لتعبر بها عن العمليات الأولية التي يقوم بها الحاسب، مثل: الإضافة add ، الحفظ Store ، الطرح Sub وغيرها.

٣. لغات عالية المستوى (High level language):

هي عبارة عن لغة يتم كتابتها باللغة الإنجليزية ويتم تشغيلها وتنفيذها في الحاسب الآلي من خلال محول أو مفسر؛ لتحويلها وتنفيذها بلغة الآلة وتعتبر سهلة التعلم مقارنة باللغات السابقة، ومن أمثلتها لغة: Java ، C# ، Python ، وهي اللغة التي سوف نتعلمها في هذا الحقيبة.



الشكل التالي يوضح مستويات لغات البرمجة في الحاسب الآلي:



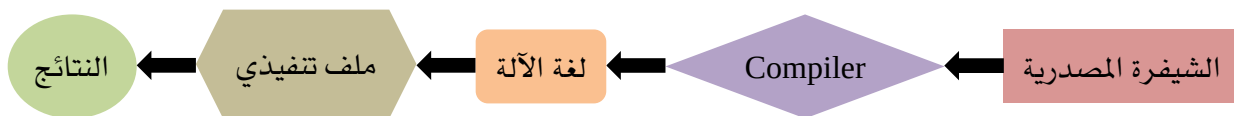
الشكل (١ - ٢)

الفرق بين المترجم The compiler والمفسر The Interpreter:

المترجم والمفسر يقومان بنفس العمل، وهو تحويل لغات عالية المستوى إلى لغة الآلة، لكن يختلفان في طريقة التنفيذ، من هنا سوف نبدأ في التعرف على كل واحد على حده؛ ليسهل عليك فهمهما بسهولة:

أولاً: المترجم The Compiler:

يقوم المترجم The Compiler بتحويل الشيفرة المصدرية (البرمجة) إلى ملف تنفيذي (An Executable File)، حيث يتم تنفيذه مباشرة بواسطة المعالج ويحمل في ذاكرة الحاسب الآلي.



الشكل (١ - ٣)

ثانياً: المفسر The Interpreter:

يقوم المفسر The Interpreter بتحويل الشيفرة المصدرية (البرمجة) بواسطة المفسر إلى لغة الآلة مباشرة، حيث يقوم المفسر بتنفيذ البرمجة في آلة افتراضية تحاكي عمل الحاسب الآلي.



الشكل (١ - ٤)



معلومات مهمة:

- أغلب البرامج الموجودة في الإنترنت والأكثر مبيعاً منفذة بواسطة المترجم The Compiler؛ لأنها أسرع بالتنفيذ بسبب تشغيلها على معالج وذاكرة الجهاز مباشرة.
- برامج المفسر The Interpreter تتميز بخاصية التنقل The portability، أي: سهولة تشغيل البرمجة من نظام تشغيل إلى آخر دون كتابة برنامج لكل نظام تشغيل، بعكس المترجم تحتاج إلى إنشاء ملف تنفيذي لكل نظام تشغيل.

جدول يبين المقارنة بين المترجمات والمفسرات:

وجه المقارنة	المترجمات	المفسرات
مبدأ العمل	تحليل كامل للشفيرة المصدرية، وتحسينها وتوليد البرنامج التنفيذي.	تنفيذ الشيفرة المصدرية بشكل مباشر سطرًا تلو الآخر.
دخل	كامل الشيفرة المصدرية.	سطر واحد من الشيفرة المصدرية.
الخرج	برنامج تنفيذي بلغة الآلة.	تنفيذ مباشر للشفيرة المصدرية.
أشهر اللغات	C, C++, C#, Scala	Python, Ruby, PHP

مقدمة لمفهوم الخوارزميات وحل المشكلات بالحاسب الآلي:

قبل أن نتعلم لغة البرمجة، لابد أن تستوعب أن البرمجة عبارة عن لغة يتم بواسطتها ترجمة فكرة أو حل مشكلة عن طريق الحاسب الآلي، فدون إيجاد الحل أو الفكرة لا يمكن كتابة البرنامج، وكيفية تعلم البرمجة لابد أن تفهم علم الخوارزميات وخرائط التدفق.

الخوارزميات:

هي عبارة عن مجموعة من الخطوات والتعليمات الرياضية والمنطقية المتسلسلة التي تساعد في حل مشكلة أو إنجاز مهمة معينة.

فالخوارزمية لها طريقة كتابة لابد من اتباعها؛ لتكون مفهومة عند المبرمج:

١. البداية.

٢. محتوى الخوارزمية.

٣. النهاية.



مثال على الخوارزمية: عمل كوب من الشاي.

خطوات حل الخوارزمية:

١. البداية.
٢. تحديد الأدوات المطلوبة لعمل الشاي: (ماء، وعاء للغلي، كوب، شاي، سكر، ملعقة).
٣. سكب الماء في وعاء الغلي.
٤. وضع الشاي والسكر في الكوب.
٥. سكب الماء المغلي في الكوب.
٦. تحريك الشاي والسكر بواسطة الملعقة.
٧. النهاية.

كما نلاحظ في الخوارزمية السابقة كلمة (البداية والنهاية) تكون ثابتة عند كتابة الخوارزمية.

مثال للخوارزمية الرياضية:

إدخال عددين وطباعة حاصل مجموعهما عندما يكون أكبر من ٦.

خطوات حل الخوارزمية:

١. البداية.
٢. إدخال عددين من قبل المستخدم A و B.
٣. جمع $A+B$.
٤. مقارنة مجموع A و B بالرقم ٦ للتأكد من الشرط.
٥. إعادة المحاولة في حالة عدم توفر الشرط أو طباعة النتائج في حالة توفر الشرط.
٦. النهاية.

كما تلاحظ في الخوارزمية السابقة سوف تتوفر فيها ثلاث عمليات أساسية: (الاختيار والتسلسل والتكرار)، حيث تم اختيار قيم A و B ثم التسلسل بجمع القيم ثم مقارنتها بالرقم ٦ وفي الأخير القيام بعملية التكرار عند عدم تحقق الشرط.



الحل:

١. البداية.
٢. إدخال عددين من قبل المستخدم ٢ و ٣.
٣. جمع ٢+٣.
٤. مقارنة المجموع ٥ بالرقم ٦ للتأكد من الشرط.
٥. إعادة المحاولة في حالة عدم توفر الشرط أو طباعة النتائج في حالة توفر الشرط.
٦. النهاية.

خرائط التدفق Flowchart:

هي تمثيل بياني لتدفق البيانات يعتمد على رموز معروفة لتوضيح ترتيب وتسلسل الخطوات اللازمة لحل المشكلة.

مميزات خرائط التدفق:

- الاتصال:** حيث إن خرائط التدفق مكونة من أشكال نمطية فإنها تمثل وسيلة سهلة لشرح الخطوات للآخرين.
- تحليل فعال:** يمكن تحليل المسألة بصورة أكثر فاعلية باستخدام خرائط التدفق لتصبح أكثر وضوحاً.
- توثيق صحيح:** تعتبر خرائط التدفق لحل المشكلة من الأدوات الهامة لتوثيق الحل.

عيوب خرائط التدفق:

- أسلوب معقد:** تكون خرائط التدفق أكثر تعقيداً عندما تكون المسألة المراد حلها معقدة.
- إجراء تعديلات:** تحتاج خرائط التدفق إلى إعادة التمثيل عند التعديل على حل المشكلة.



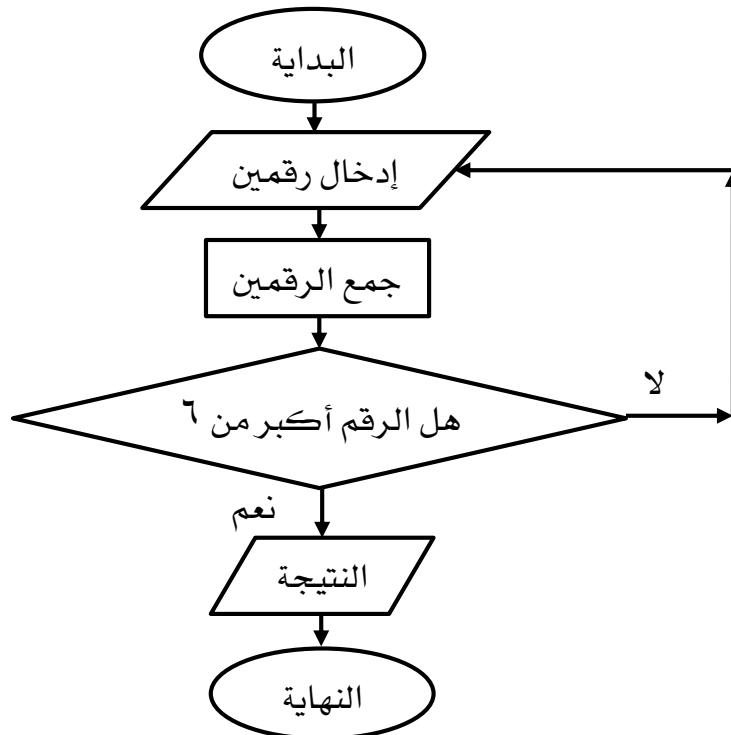
رموز خرائط التدفق:

هنالك رموز شائعة ومحددة تستخدم عند رسم خرائط التدفق وهي:

الأطراف	
تستخدم في الأطراف حيث تبين (البداية والنهاية)	
العمليات	
يمثل جميع العمليات مثل: الضرب، الجمع، القسمة ... إلخ	
المدخلات والمخرجات	
يتم تمثيل المدخلات والمخرجات بهذا الشكل	
القرارات	
يتم تمثيل القرارات والشروط بهذا الشكل	
الأسهم	
تستخدم الأسهم لتوضيح تسلسل العمليات والانتقالات	

مثال على خرائط التدفق:

ادخل عددين واطبع مجموعهما، إذا كان الناتج أكبر من الرقم ٦.



الشكل (١ - ٥)



حل المشكلات :

عبارة عن عملية منظمة تتضمن سلسلة من الخطوات المتبعة لتحقيق الهدف أو إزالة المشكلة، ويمكن اتباع الخطوات التالية للحصول على حل للمشكلة:

- تحديد المشكلة: يكون بالاعتراف بوجود مشكلة.
- تمثيل المشكلة: فهم خاص للمشكلة وإمكانية حلها.
- جمع المعلومات: يكون من المعرفة السابقة أو الذاكرة.
- توليد الحل: إيجاد الحل بالوسائل المتاحة من معطيات المشكلة.
- تنفيذ الحل: يبدأ باختيار استراتيجية تحقيق الهدف وينتهي بالحكم على فاعلية الاستراتيجية في تحقيق الهدف.
- تقويم الحل: التقويم لابد أن يكون قبل البدء لتلافي الأخطاء و بعد التنفيذ في ضوء أطر عمل ثابتة ومنظمة.

جمل التحكم بتنفيذ البرامج :

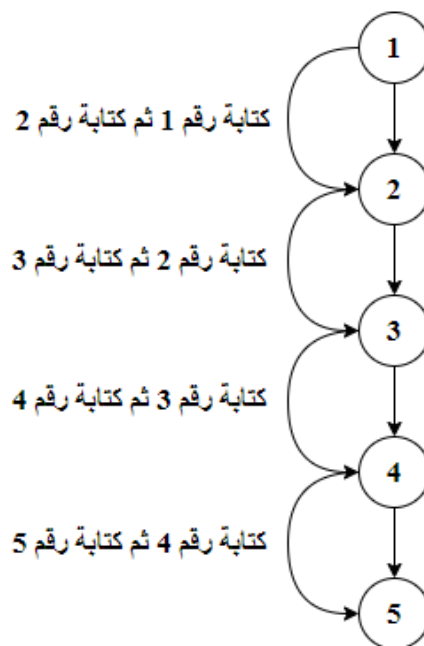
هناك عنصران أساسيان في برمجة برامج الحاسب الآلي هي: البيانات والهيكلية، وللتعامل مع البيانات لابد من فهم المتغيرات وأنواع البيانات، أما بالنسبة للهيكلية لابد من فهم صيغ وتركيبات التحكم. حيث يمكن التحكم في تنفيذ برامج الحاسب الآلي وفق ثلاثة معايير أساسية:

١. التسلسل sequence:

يتم تنفيذ البرنامج وفق مجموعة أوامر متسلسلة، حيث كل أمر يتبع الأمر الذي يليه ولا يمكن تنفيذ أي أمر قبل الآخر حسب التسلسل.

مثال:

كتابة الأرقام من 1 إلى 5 تسلسلياً من الأصغر إلى الأكبر.



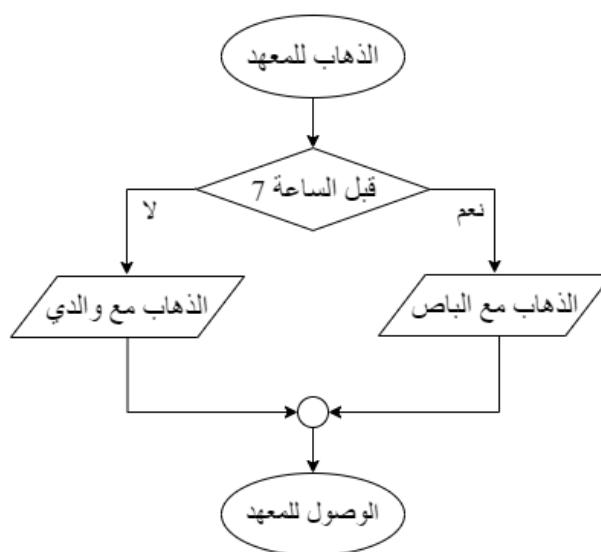
الشكل (١ - ٦)

٢. الاختيار selection:

يتم تنفيذ البرنامج من خلال تحديد خيارين فقط ولا يتم إكمال البرنامج إلا بعد تحقق أحد الخيارين ثم يتم إكمال عمل البرنامج وفق التعليمات المدخلة.

مثال:

الذهاب للمعهد قبل الساعة السابعة مع الباص وبعد الساعة السابعة مع والدي.



الشكل (١ - ٧)

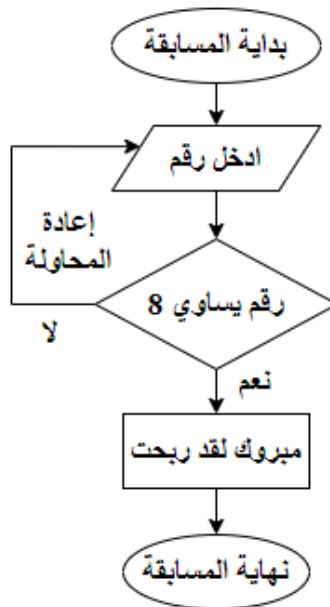


٣. التكرار iteration:

هو تكرار عدد من الأوامر أو الأحداث المحددة من المبرمج لتنفيذ جزء أو كامل البرنامج، حيث لا يتم التنفيذ إلا بعد إكمال التكرار.

مثال:

مسابقة رقم الحظ ، حيث يتم الطلب من المستخدم إدخال رقم، فإذا كان الرقم المدخل يساوي 8 يفوز المستخدم في المسابقة، وإلا تتم إعادة تكرار المسابقة.



الشكل (١ - ٨)



تمارين الوحدة

١. وضح خطوات عمل الحاسب الآلي.
٢. اذكر تعريف البرنامج.
٣. اذكر أنواع لغات البرمجة.
٤. قم بكتابة خوارزمية للحالات التالية:
 - تشغيل جهاز حاسب آلي جديد.
 - الذهاب للمعهد صباحاً.
٥. قم بتمثيل الحالات التالية باستخدام خرائط التدفق:
 - الخطوات التي يمر بها اتصال هاتفي، عندما يصلك اتصال فإنك تتظر في الهاتف ثم تقرر ما إذا كنت سترد أم لا؟، ففي حالة عدم الرد فإنك تضع الهاتف جانباً أما في حالة الرد فإنك ترد ثم تضع الهاتف جانباً.
 - برنامج يقوم بإدخال العمر ثم مقارنة عمرك مع الرقم ١٥ ففي حالة كان عمرك أكبر أو يساوي رقم ١٥ يقوم بطباعة الجملة: (أهلاً بك في لغة بايثون)، أما في حالة كان عمرك أصغر من ١٥ يقوم بطباعة الجملة: (عذراً لا تستطيع الدخول في لغة بايثون) ثم ينتهي البرنامج.
٦. قم بإيجاد حل للمشاكل التالية:
 - ضياع مفتاح الباب.
 - عدم عمل جهاز حاسب آلي.
٧. اذكر أنواع جمل التحكم في تنفيذ البرامج مع رسم مخطط لكل نوع؟



نموذج تقييم المتدرب لمستوى أدائه					
يعبأ من قبل المتدرب نفسه وذلك بعد الانتهاء من تمارين الوحدة					
بعد الانتهاء من التدريب على وحدة مقدمة في البرمجة ولغاتها، قيم نفسك وقدراتك بواسطة إكمال هذا التقييم الذاتي بعد كل عنصر من العناصر المذكورة، وذلك بوضع علامة (✓) أمام مستوى الأداء الذي أتقنته، وفي حالة عدم قابلية المهمة للتطبيق ضع العلامة في الخانة الخاصة بذلك.					
م	العناصر	مستوى الأداء (هل أتقنت الأداء)			
		غير قابل للتطبيق	لا	جزئيا	كليا
١	يحل المسائل باستخدام الخوارزميات.				
٢	يمثل الخوارزميات بخرائط التدفق في المسائل.				
٣	يتحكم في تنفيذ برامج الحاسب الآلي.				
يجب أن تصل النتيجة لجميع المفردات (البنود) المذكورة إلى درجة الإتقان الكلي أو غير قابلة للتطبيق، وفي حالة وجود مفردة في القائمة "لا" أو "جزئيا" فيجب إعادة التدريب على هذا النشاط مرة أخرى بمساعدة المدرب.					



نموذج تقييم المدرب لمستوى أداء المتدرب					
يعبأ من قبل المدرب وذلك بعد الانتهاء من تمارين الوحدة					
اسم المتدرب :		التاريخ :			
رقم المتدرب :		المحاولة : ١ ٢ ٣ ٤			
		العلامة :			
كل بند أو مفردة يقيم بـ ١٠ نقاط الحد الأدنى: ما يعادل ٨٠٪ من مجموع النقاط. الحد الأعلى: ما يعادل ١٠٠٪ من مجموع النقاط.					
م	بنود التقييم	النقاط (حسب رقم المحاولات)			
		١	٢	٣	٤
١	يحل المسائل باستخدام الخوارزميات.				
٢	يمثل الخوارزميات بخرائط التدفق في المسائل.				
٣	يتحكم في تنفيذ برامج الحاسب الآلي.				
المجموع					
ملحوظات:					
توقيع المدرب:					



الوحدة الثانية

مقدمة إلى لغة بايثون



الوحدة الثانية

مقدمة إلى لغة بايثون

الهدف العام للوحدة:

تهدف هذه الوحدة إلى تعريف المتدرب بلغة برمجة بايثون وطريقة تثبيتها واستخدامها.

الأهداف التفصيلية:

من المتوقع في نهاية هذه الوحدة التدريبية أن يكون المتدرب قادراً وبكفاءة على أن:

١. يثبت لغة بايثون على جهاز الحاسب الآلي.
٢. يستخدم محرر الكتابة في لغة بايثون.
٣. يكتب برنامج باستخدام الدالة.
٤. يكتب التعليقات في لغة بايثون.

الوقت المتوقع للتدريب على هذه الوحدة: ١٠ ساعات تدريبية.

الوسائل المساعدة:

١. جهاز حاسب آلي.
٢. جهاز عرض (Data Show).
٣. اتصال إنترنت.
٤. محرر كتابة Python.



لغة بايثون Python:

هي لغة برمجة من لغات المستوى العالي، والتي تتميز ببساطة كتابتها وقراءتها وقابليتها للتطوير، وهي لغة تفسيرية تستخدم أسلوب البرمجة الكائنية.

مميزات لغة بايثون:

١. سهولة التعلم.
٢. حرة ومفتوحة المصدر.
٣. لغة برمجة عالية المستوى.
٤. متعددة الاستخدامات.
٥. تدعم جميع أنظمة التشغيل مثل Windows، Linux، macOS.

استخدامات لغة بايثون:

١. يمكن الاستفادة من هذه اللغة في أمن المعلومات.
٢. يمكنها التعامل مع الشبكات.
٣. يمكن من خلالها تصميم صفحات ويب.
٤. يمكن استخدامها لتصميم برامج لنظام الأندرويد.
٥. يمكن استخدامها في تصميم برامج سطح المكتب.

تنويه: لغة البرمجة بايثون تعتمد على المفسر The Interpreter وهذا سبب توافقها مع جميع أنظمة التشغيل.

المقصود ببرمجة كائنية التوجه - Object-oriented programming:

عبارة عن نمط برمجة حيث يمكن استخدامه تقسيم البرامج إلى وحدات تسمى الكائنات Objects، حيث يكون كل كائن عبارة عن حزمة من البيانات. ويتم البرمجة بواسطة استخدام الكائنات وربطها مع بعضها مع واجهة البرنامج الخارجية باستخدام هيكلية البرنامج وواجهات الاستخدام الخاصة بكل كائن.



التعرف على أفضل محررات بايثون:

يوجد العديد من المحررات التي تدعم نظام بايثون ومن أشهرها:

- Spyder Python IDE
- PyCharm Python IDE
- Thonny Python IDE
- Python IDE: وهو المحرر الذي سوف يتم استخدامه في شرح هذا الكتاب.

تثبيت لغة بايثون على الجهاز:

ملاحظة: الخطوات التالية لنظام التشغيل Windows فقط.

التأكد من وجود بايثون على الجهاز:

يتم التأكد من وجود بايثون على جهاز الحاسب الآلي من خلال الخطوات التالية:

١. فتح موجه الأوامر cmd.

٢. كتابة الأمر التالي: python ثم الضغط على مفتاح Enter.

الرسالة التالية تعني وجود بايثون على الجهاز فلا تحتاج لتثبيته، لكن نحتاج لتثبيت المحرر

وذلك لسهولة كتابة الشيفرة البرمجية.

```
C:\Users\Tam>python
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 21:26:53) [MSC v.1916 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
```

الشكل (٢ - ١)

في حالة عدم وجود بايثون على الجهاز وهذا من المستبعد، يتم تثبيت بايثون من خلال

إحدى الطريقتين التاليتين:

أولاً: استخدام أداة الحزم التي تأتي مع النظام لتثبيت حزمة بايثون.

ثانياً: تحميل محرر لغة بايثون من موقع Python:

- من خلال الدخول على موقع الشركة الرسمي python.org.
- الاطلاع على الصورة التالية.



الشكل (٢ - ٢)

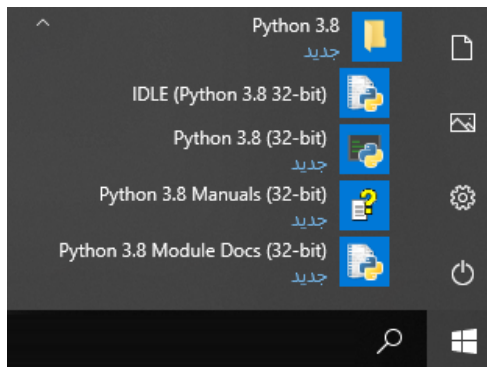
تشبيت محرر لغة بايثون على الجهاز:

اتباع الخطوات كما في الصورة التالية:



الشكل (٢ - ٣)

تشغيل محرر كتابة بايثون Python IDLE:



الشكل (٢ - ٤)

- الضغط على قائمة ابدأ.
- البحث عن مجلد Python 3.8.
- اختيار Python 3.8 IDLE.



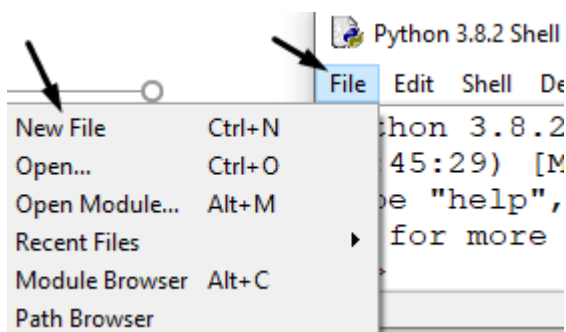
التعرف على بيئة محرر كتابة بايثون Python IDLE:

الصورة التالية: توضح واجهة ال Shell في محرر IDLE-Python، حيث يتم كتابة الأوامر مباشرة دون كتابة كود برمجي.

```
File Edit Shell Debug Options Window Help
Python 3.8.2 (tags/v3.8.2:7b3ab59, Feb 25 2020,
22:45:29) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license(
)" for more information.
>>> | ← يتم كتابة الأوامر هنا مباشرة
```

الشكل (٢ - ٥)

فتح ملف جديد:

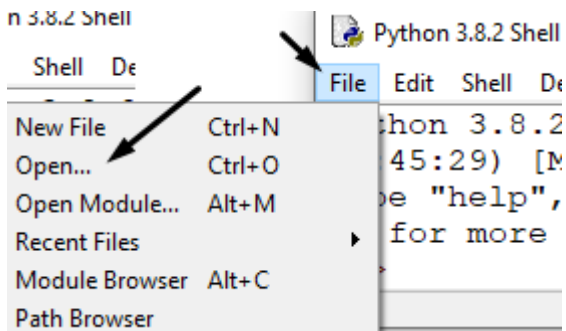


١. فتح قائمة File.

٢. اختيار New File.

الشكل (٢ - ٦)

فتح ملف محفوظ:



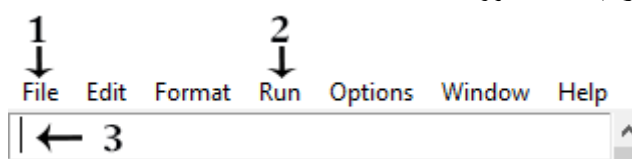
١. فتح قائمة File.

٢. اختيار Open.

٣. تحديد مكان الملف المحفوظ.

الشكل (٢ - ٧)

شرح أهم الخيارات في واجهة محرر IDLE:

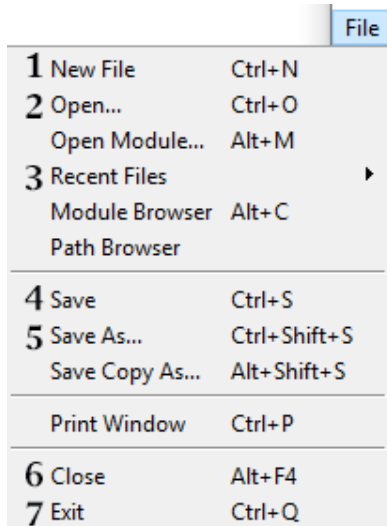


الشكل (٢ - ٨)



١. File: قائمة ملف، حيث يوجد فيها عدة خيارات منها: فتح، حفظ الملف.
٢. Run: تسمح بتجربة البرنامج بعد الانتهاء من كتابة الكود، بالإضافة لتشغيل Shell.
٣. Coding Place: مساحة العمل، حيث يتم من خلالها كتابة الشيفرة البرمجة.

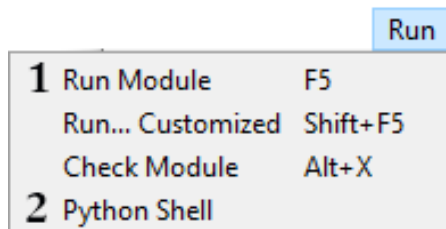
شرح قائمة ملف:



١. إنشاء ملف جديد.
٢. فتح ملف محفوظ.
٣. فتح آخر الملفات المستخدمة.
٤. حفظ الملف.
٥. حفظ الملف باسم.
٦. إغلاق الملف.
٧. إغلاق المحرر.

الشكل (٢ - ٩)

شرح قائمة تشغيل:



١. تنفيذ ملف البرمجة.
٢. فتح الـ Shell.

الشكل (٢ - ١٠)

لاحقة حفظ الشيفرة البرمجية في لغة بايثون.

يتم حفظ ملف الشيفرة البرمجية في لغة بايثون باللاحقة (.py).

example.py

أساسيات في لغة بايثون:

١. أساليب كتابة الشيفرة البرمجية.

في لغة بايثون يمكن كتابة كل أمر برمجي في سطر مستقل أو عدة أوامر برمجية في سطر واحد وذلك من خلال الفصل بينهم بعلامة (;).



مثال: كتابة كل أمر برمجي في سطر مستقل:

```
print('TVTC') ← 1
print(2002) ← 2
```

النتيجة:

```
TVTC
2002
```

في المثال: تمت كتابة الأمر الأول في سطر مستقل ثم النزول لسطر جديد وكتابة الأمر الثاني.

مثال: كتابة أكثر من أمر برمجي في سطر واحد:

```
print('TVTC'); print(2002)
    ↑           ↑
    1           2
```

النتيجة:

```
TVTC
2002
```

في المثال: تمت كتابة الأمر الأول ثم إضافة علامة (;) ثم كتابة الأمر الثاني في نفس السطر.

٢. الدالة.

عبارة عن مجموعة من الأوامر تعمل مع بعض وفق تسلسل معين، وتنفذ عند استدعائها من المبرمج، وفي لغة بايثون الدوال تنقسم لقسمين:

- ١ دوال جاهزة وسوف نتعرف عليها في المواضيع القادمة
- ٢ دوال يتم إنشائها بواسطة المبرمج وسوف نتعرف عليها لاحقاً.

٣. تعريف القيم النصية في لغة بايثون.

باستخدام علامة التنصيص (الفردية أو المزدوجة)، يتم تعريف القيم النصية في لغة بايثون، وتحويل أي قيمة (رقمية أو منطقية) إلى نصية.

```
'Hello, World!'
"Hello, World!"
'2020'
"1441"
```



٤. الكلمات المحجوزة في بايثون.

هي كلمات لا يمكن استخدامها في لغة بايثون عند تعريف متغير أو كلاس أو دالة أو كائن إلخ ، وسيتم شرح كل من هذه المصطلحات في وقتها. وفي الجدول التالي أمثلة على أشهر الكلمات المحجوزة في بايثون:

return	if	def	not	exec	and
finally	import	del	or	pass	assert
while	except	elif	try	raise	break
global	continue	else	is	from	class
yield	lambda	in	for	with	print

دالة print():

١. كتابة أول برنامج باستخدام دالة print().

دالة print(): وهي عبارة عن دالة جاهزة تقوم بطباعة رسالة معينة على الشاشة، حيث من الممكن أن تكون الرسالة عبارة عن نصوص أو أرقام أو أي عنصر محدد من قبل المبرمج.

```
print(value)
  ↑   ↑   ↑
  1   2   3
```

١. print: اسم الدالة.

٢. (:): قوس الدالة، (يفتح في البداية ويغلق في النهاية).

٣. value: القيمة المطبوعة سواء: (نصية أو رقمية إلخ)

مثال: كتابة برنامج يطبع جملة (Hello, World!).

```
print('Hello, World!')
```

النتيجة:

```
Hello, World!
```

في المثال: تمت طباعة القيمة (Hello, World!) باستخدام دالة print.

تنبيه: دالة print() تقوم بالتعامل مع الأرقام كآلة حاسبة حيث تقوم بتنفيذ العمليات الحسابية مباشرة بمجرد كتابتها بين الأقواس.



```
print(3*5)
```

النتيجة:

```
15
```

في المثال: تم كتابة العملية الحسابية (٥*٣) في دالة print، وتم طباعة الناتج ١٥.

٢. معاملات تستخدم مع دالة print():

- طباعة أكثر من ناتج دالة print () في سطر واحد.

```
print(value, end='separator')
```

↑ ↑ ↑ ↑ ↑ ↑ ↑
1 2 3 4 5 6 7 8

١. print: اسم الدالة.

٢. (:): قوس الدالة، (يفتح في البداية ويغلق في النهاية).

٣. value: القيمة المطبوعة سواء: (نصية أو رقمية إلخ)

٤. (,): علامة الفاصلة توضع بين القيمة وكلمة end.

٥. end: اسم المعامل المستخدم.

٦. (=): علامة يساوي توضع بين المعامل وعلامة تنيص الفاصل.

٧. (' '): علامة التنصيص يمكن استخدام المفردة أو المزدوجة؛ لتحديد الفصل المطلوب.

٨. separator: الفاصل المطلوب استخدام.

```
print('Hello', end='')  
print('Python')
```

النتيجة:

```
HelloPython
```

في المثال: لم يتم تحديد الفاصل بين دالتي الطباعة، فكانت النتيجة

طباعة القيمتين بجانب بعض.

```
print('Hello', end='@')  
print('Python')
```

النتيجة:

```
Hello@Python
```

في المثال: تم تحديد الفاصل بين دالتي الطباعة بـ (@)، فكانت النتيجة

إضافة علامة (@) بين القيمتين المطبوعتين.



تحديد فاصل محدد بين القيم المطبوعة في دالة print().

```
print(value, sep="separator")
```

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
1 2 3 4 5 6 7 8

١. print : اسم الدالة.
٢. (): قوس الدالة ، (يفتح في البداية ويغلق في النهاية).
٣. value : القيمة المطبوعة سواء: (نصية أو رقمية إلخ)
٤. (,): علامة الفاصلة توضع بين القيمة وكلمة end.
٥. sep : اسم المعامل المستخدم.
٦. (=): علامة يساوي توضع بين المعامل وعلامة تنصيب الفاصل.
٧. (' '): علامة التنصيب يمكن استخدام المفردة أو المزدوجة؛ لتحديد الفصل المطلوب.

٨. separator : الفاصل المطلوب استخدامه.

```
print('T', 'V', 'T', 'C', sep="")
```

النتيجة:

TVTC

في المثال: لم يتم تحديد الفاصل بين القيم المطبوعة في دالة print، فكانت النتيجة طباعة القيم بجانب بعضها.

```
print('T', 'V', 'T', 'C', sep="-")
```

النتيجة:

T-V-T-C

في المثال: تم تحديد الفاصل بين القيم المطبوع في دالة print بـ (-)، فكانت النتيجة طباعة القيم مع إضافة علامة (-) بينها.

مهارات في دالة print():

١. استخدام علامة التنصيب مع النصوص.

• استخدام علامة التنصيب المزدوجة مع المفردة.

```
print('Hello, "world"')
```

Hello, "world"

النتيجة:



في المثال: تم تعريف النص بعلامة التنصيص المفردة (' ')، واستخدام علامة التنصيص المزدوجة لتحديد كلمة (world).

- استخدام علامة التنصيص المفردة مع المزدوجة.

```
print("I'm useing Python")
```

النتيجة:

```
I'm useing Python
```

في المثال: تم تعريف النص بعلامة التنصيص المزدوجة (" "), واستخدام

علامة التنصيص المفردة لاختصار كلمة (I'm).

- استخدام علامة التنصيص المفردة مع المفردة.

```
print('I\'m useing Python')
```

النتيجة:

```
I'm useing Python
```

في المثال: تم تعريف النص بعلامة التنصيص المفردة (' '), وتم إضافة

علامة (\) قبل علامة أي تنصيص مفردة داخل النص.

تنويه: علامة (\) تعرف باسم (backslash) وتستخدم عند وجود علامة

تنصيص في النص المعرف بعلامة تنصيص فردية.

٢. استخدام علامة (+) في دالة print():

- مع النصوص:

```
print('Hi'+'Python')
```

النتيجة:

```
HiPython
```

في المثال: عند استخدام علامة (+) مع النصوص تتم الطباعة بجانب بعضهم

بدون مسافة.

- مع الأرقام:

```
print(5+5)
```

النتيجة:

```
10
```

في المثال: عند استخدام علامة (+) مع الأرقام يتم جمع الأرقام وطباعة الناتج.



٣. استخدم علامة (*) في دالة print():

• مع النصوص:

```
print('Hi'*3)
```

النتيجة:

```
HiHiHi
```

في المثال: عند استخدام علامة (*) مع النصوص يتم تكرار النص بعدد مرات الرقم المحدد، حيث تم تكرار كلمة (Hi) ثلاث مرات؛ لأنها ضربت في رقم (٣).

• مع الأرقام:

```
print(3*3)
```

النتيجة:

```
9
```

في المثال: عند استخدام علامة (*) مع الأرقام يتم ضرب الأرقام وطباعة الناتج.

٤. طباعة أكثر من سطر في دالة print():

باستخدام علامة التنصيص الثلاثية (") تستطيع طباعة أكثر من سطر.

```
1 2 3
↓ ↓ ↓
print(''''
line
line ← 4
line
...etc
''')
```

١. print: اسم الدالة.

٢. (: قوس الدالة، (يفتح في البداية ويغلق في النهاية).

٣. ("): علامات التنصيص الثلاثة، قبل وبعد الأسطر، (يستخدم مفردة أو مزدوجة)

٤. line: مكان كتابة الأسطر.

```
print(''''Definitions of "TVT(
T:Technical
V:Vocational
T:Training
C:Corporation''')
```



النتيجة :

```
Definitions of "TVTC":
T:Technical
V:Vocational
T:Training
C:Corporation
```

٥. وضع مسافة Tab (٥ مسافات):

يمكن إضافة مسافة Tab بين القيم الموجودة في دالة print()، وذلك باستخدام (t)

```
print('value1\tvalue2')
```

↑ ↑↑ ↑ ↑ ↑
1 2 3 4 5 6

١. print: اسم الدالة.

٢. (:): قوس الدالة، (يفتح في البداية ويغلق في النهاية).

٣. (''): علامة التنصيص يمكن استخدام المفردة أو المزدوجة؛ لتحديد الفصل المطلوب.

٤. Value1: القيمة المطبوعة سواء: (نصية أو رقمية إلخ)

٥. (\t): الشرطة المعكوسة مع حرف (t) بين القيمتين المطلوب وضع المسافة بينهما.

٦. Value2: القيمة المطبوعة سواء: (نصية أو رقمية إلخ)

```
print('Hello,\tWorld!')
```

النتيجة :

```
Hello,    World!
```

في المثال: تم إدراج العلامة (\t) بين كلمة (Hello,) وكلمة (World!)، حيث تمت

طباعة الكلمتين بإضافة مسافة بمقدار Tab (٥ مسافات)، بالرغم من وجودهم بنفس

علامة التنصيص.

٦. طباعة أكثر من قيمة في دالة print():

```
print(value1,value2,value3...etc)
```

↑ ↑ ↑ ↑ ↑
1 2 3 4 5

١. print: اسم الدالة.

٢. (:): قوس الدالة، (يفتح في البداية ويغلق في النهاية).

٣. value1: القيمة الأولى.



٤. (,) : علامة الفاصلة توضع بين القيم.

٥. value2 : القيمة الثانية ، ويتم الاستمرار بوضع الفاصلة بين كل قيمة.

```
print('a','b',2020)
```

النتيجة :

```
a b 2020
```

في المثال: تمت طباعة ثلاث قيم مختلفة النوع باستخدام دالة print واحدة، حيث تم الفصل بين القيم بعلامة الفاصلة (,).

٧. النزول سطر جديد في دالة print().

يمكن طباعة القيم الموجودة في دالة print() في أكثر من سطر وذلك باستخدام (\n).
● داخل النصوص:

```
print('value1\nvalue2')
```

```
↑ ↑↑ ↑ ↑
1 23 4 5 6
```

١. print : اسم الدالة.

٢. () : قوس الدالة ، (يفتح في البداية ويغلق في النهاية).

٣. (') : علامة التنصيص يمكن استخدام المفردة أو المزدوجة ، لتحديد الفصل المطلوب.

٤. Value1 : القيمة المطبوعة سواء : (نصية أو رقمية إلخ)

٥. (\n) : الشرطة المعكوسة مع حرف (n) بين القيمتين المطلوب وضع سطر بينهما.

٦. Value2 : القيمة المطبوعة سواء (نصية أو رقمية إلخ)

```
print('Hello,\nWorld!')
```

النتيجة :

```
Hello,
World!
```

في المثال: تم إدراج العلامة (\n) بين كلمة (Hello) وكلمة (World) ، حيث تم طباعة كل كلمة في سطر مستقل، بالرغم من وجودهم بنفس علامة التنصيص ونفس دالة الطباعة.



• خارج النصوص:

```
print(value1, '\n', value2)
```

↑ ↑ ↑ ↑ ↑ ↑
1 2 3 4 5 6

١. print: اسم الدالة.

٢. (): قوس الدالة، (يفتح في البداية ويغلق في النهاية).

٣. Value1: القيمة المطبوعة سواء: (نصية أو رقمية إلخ)

٤. (,): علامة الفاصلة توضع بين القيمة وعلامة ('\n').

٥. ('\n'): الشرطة المعكوسة مع حرف (n) بين علامتي تنصيص بين القيمتين المطلوب

وضع سطر بينهما.

٦. Value2: القيمة المطبوعة سواء (نصية أو رقمية إلخ)

```
print(100, '\n', 200)
```

النتيجة:

```
100
200
```

في المثال: تم إدراج العلامة ('\n') بين رقمين (١٠٠) ورقم (٢٠٠)، حيث تمت طباعة

كل رقم في سطر مستقل، بالرغم من وجودهم بنفس دالة الطباعة.

كتابة التعليقات في لغة بايثون:

تسمح لغة بايثون بإضافة التعليقات للشفيرة البرمجية، حيث يستخدم المبرمجون التعليقات

في لغات البرمجة لشرح آلية عمل شيفرة معينة أو توضيح شيفرة برمجية، حيث يتم عرض التعليق

في كود المصدر دون تنفيذه عند تنفيذ البرنامج.

• كتابة تعليق من سطر واحد.

في لغة بايثون علامة (#) رمز الشباك، يتيح كتابة التعليقات في سطر واحد فقط،

بشرط إدراج الرمز في بداية السطر ثم كتابة التعليق.

```
#Comments
```

↑ ↑
1 2



١. (#): رمز الشباك، يتم إضافته قبل كتابة التعليق، (بداية سطر التعليقات).

٢. Comments: التعليقات، حيث يتم كتابة التعليق المطلوب.

مثال:

```
# برنامج يطبع الاسم
print('Python')
```

النتيجة:

```
Python
```

في المثال: تم طباعة كلمة (Python) فقط دون طباعة التعليق المكتوب بالأحمر.

• كتابة تعليق من عدة أسطر.

في لغة بايثون علامة التنصيص الثلاثية (' ' '): الفردية أو الزوجية، تتيح كتابة

التعليقات في أكثر من سطر، بشرط إضافة ثلاث علامات في بداية النص وفي نهايته.

```
''' ← 1
Comments ← 2
'''
```

١. (""): علامة التنصيص الثلاثية، تدرج في بداية ونهاية التعليق.

٢. Comments: التعليقات، يتم كتابتها بين علامتي التنصيص الثلاثية.

مثال

```
'''
برنامج يطبع
الاسم
العمر
'''
print('Thamer')
print('37')
```

النتيجة:

```
Thamer
37
```

في المثال: تم طباعة الاسم و العمر بدون طباعة وصف البرنامج .



تمارين الوحدة

١. اذكر أربعاً من مميزات لغة بايثون.
٢. اذكر خطوات التأكد من احتواء windows 10 على لغة بايثون.
٣. اذكر ثلاثة أنواع من محررات لغة بايثون.
٤. اختر الإجابة الصحيحة:
 لعرض النص Hello World على الشاشة نستخدم الدالة:
 - `rtscn("Hello World")`
 - `print(Hello World)`
 - `print()"Hello World"`
 - `print("Hello World")`
 عند تنفيذ `print("hello world !")` يكون الناتج:
 - `("hello world !")`
 - `"hello world !"`
 - `hello world !`
 عند تنفيذ `print(5*5)` يكون الناتج:
 - `5*5`
 - `"5*5"`
 - `25`
٥. قم بكتابة كود لعرض جملة (المؤسسة العامة للتدريب التقني والمهني) على الشاشة.
٦. قم بكتابة كود لطباعة الأرقام التالية على الشاشة: ١٤٤١ ، ١٤٤٢.
٧. قم بكتابة برنامج يقوم بعرض ناتج العملية $3 \times 5 / 3$.
٨. قم بكتابة كود يقوم بعرض جملة hello world مع إضافة تاريخ اليوم كتعليق.
٩. أجب بعلامة (✓) أمام الجملة الصحيحة وعلامة (✗) أمام الجملة غير الصحيحة:
 - تتم طباعة النصوص باستخدام دالة `print()` وتحديد النص بعلامة " " ()
 - عند طباعة النصوص والمتغيرات لا نحتاج لعلامة " " ()



- عند كتابة التعليقات نقوم بوضع علامة @ في بداية التعليق وعلامة # للنهاية ()
- نستطيع كتابة التعليق في أكثر من سطر باستخدام علامة # في البداية فقط ()

١٠. قم بكتابة البرامج التالية باستخدام دالة print():

- برنامج يقوم بعرض (لغة بايثون) على الشاشة.
- برنامج يقوم بعرض ناتج العملية $3*5$ على الشاشة.



نموذج تقييم المتدرب لمستوى أدائه					
يعبأ من قبل المتدرب نفسه وذلك بعد الانتهاء من تمارين الوحدة					
بعد الانتهاء من التدريب على وحدة مقدمة إلى لغة بايثون، قيم نفسك وقدراتك بواسطة إكمال هذا التقييم الذاتي بعد كل عنصر من العناصر المذكورة، وذلك بوضع علامة (✓) أمام مستوى الأداء الذي أتقنته، وفي حالة عدم قابلية المهمة للتطبيق ضع العلامة في الخانة الخاصة بذلك.					
م	العناصر	مستوى الأداء (هل أتقنت الأداء)			
		غير قابل للتطبيق	لا	جزئيا	كليا
١	يثبت لغة بايثون على جهاز الحاسب الآلي.				
٢	يستخدم محرر الكتابة في لغة بايثون.				
٣	يكتب برنامج باستخدام الدالة.				
٤	يكتب التعليقات في لغة بايثون.				
يجب أن تصل النتيجة لجميع المفردات (البنود) المذكورة إلى درجة الإتقان الكلي أو غير قابلة للتطبيق، وفي حالة وجود مفردة في القائمة "لا" أو "جزئيا" فيجب إعادة التدريب على هذا النشاط مرة أخرى بمساعدة المدرب.					



نموذج تقييم المدرب لمستوى أداء المتدرب					
يعبأ من قبل المدرب وذلك بعد الانتهاء من تمارين الوحدة					
اسم المتدرب :		التاريخ:			
رقم المتدرب :		المحاولة: ١ ٢ ٣ ٤			
		العلامة:			
كل بند أو مفردة يقيم بـ ١٠ نقاط الحد الأدنى: ما يعادل ٨٠٪ من مجموع النقاط. الحد الأعلى: ما يعادل ١٠٠٪ من مجموع النقاط.					
م	بنود التقييم	النقاط (حسب رقم المحاولات)			
		١	٢	٣	٤
١	يثبت لغة بايثون على جهاز الحاسب الآلي.				
٢	يستخدم محرر الكتابة في لغة بايثون.				
٣	يكتب برنامج باستخدام الدالة.				
٤	يكتب التعليقات في لغة بايثون.				
المجموع					
ملحوظات:					
.....					
توقيع المدرب:					



الوحدة الثالثة

أنواع البيانات والمتغيرات



الوحدة الثالثة

أنواع البيانات والمتغيرات

الهدف العام للوحدة:

تهدف هذه الوحدة إلى إتقان المتدرب التعامل مع البيانات والمتغيرات وبعض الدوال في لغة بايثون.

الأهداف التفصيلية:

- من المتوقع في نهاية هذه الوحدة التدريبية أن يكون المتدرب قادراً وبكفاءة على أن:
١. يوضح أنواع البيانات في لغة بايثون.
 ٢. يستخدم الدوال في لغة بايثون.
 ٣. يمثل المتغيرات في لغة بايثون.

الوقت المتوقع للتدريب على هذه الوحدة: ١٥ ساعة تدريبية.

الوسائل المساعدة:

١. جهاز حاسب آلي.
٢. جهاز عرض (Data Show).
٣. محرر كتابة Python



البيانات في لغة بايثون :

البيانات عبارة عن مدخلات يطلبها البرنامج من المستخدم أو المبرمج للعمل بشكل صحيح أو تنفيذ مهمة معينة ، وفي لغة بايثون يتم تصنيف وتخزين البيانات في عدة أنواع رئيسية تلقائياً أو عن طريق المبرمج باستخدام بعض الدوال المساعدة ، وذلك لسهولة التعامل معها.

أنواع البيانات الأساسية في بايثون:

١ . الأرقام Numbers.

٢ . النوع المنطقي Boolean type.

٣ . متسلسلات حرفية Strings.

٤ . القوائم Lists.

٥ . المصفوفات Tuples.

٦ . المجموعات Sets.

٧ . القاموس Dictionary.

لكل نوع منها خصائص واستخدامات في هذه الحقبة سوف نتعرف على ثلاثة أنواع هي:

١ . الأرقام Numbers:

هي الرموز المستخدمة للتعبير عن الأعداد الصحيحة أو العشرية ، مثل: (١ ، ٢ ، ٣ ، ٥ ، ١٤ ، ٣) ، وفي لغة بايثون تنقسم الأرقام إلى ثلاثة أقسام ، سوف نتعرف على اثنين منها:

أولاً: الأرقام الصحيحة Integers:

هي الأعداد التي تكتب بدون استخدام الكسور أو الفواصل مثل (١ ، ٢ ، ٣) ، في لغة بايثون يرمز لها بالاختصار (int) ، وتمثل بدالة int().
مثال: اكتب برنامج يطبع الرقم (٣).

```
print (3)
```

النتيجة:

```
3
```

في المثال: تمت طباعة الرقم بدون فاصلة عشرية للدلالة على أنه رقم صحيح.



مثال: اكتب برنامج يطبع الرقم (٢٠٢٠).

```
print (2020)
```

النتيجة:

```
2020
```

ثانيًا: الأرقام العشرية Floats:

هي الأعداد التي تكتب باستخدام الكسور أو الفواصل مثل: (٣,١٤,٢,٥,٠,٥)، وفي لغة بايثون يرمز لها بالاختصار (float)، وتمثل بدالة float().

مثال: اكتب برنامج يطبع الرقم (٣,٥).

```
print (3.5)
```

النتيجة:

```
3.5
```

في المثال: تمت طباعة الرقم مع فاصلة عشرية للدلالة على أنه رقم عشري.

مثال: اكتب برنامج يطبع الرقم (١٤,٤٢).

```
print (14.42)
```

النتيجة:

```
14.42
```

مقارنة بين الأرقام الصحيحة والعشرية:

المقارنات	الصحيح	العشري
مثال	2	1.5
الاسم	Integers	Float
الدالة	int()	float()
العلامة الفارقة	بدون فاصلة	يحتوي فاصلة
النوع في لغة بايثون	<class 'int'>	<class 'float'>



٢. النوع المنطقي (البولياني) Boolean:

عبارة عن بيانات قد تحمل إحدى القيمتين (صح True) أو (خطأ False) فقط، ويمكن تمثيلها بالأرقام (1 للصح) و (0 للخطأ). وفي لغة بايثون تكون عبارة عن كائنات ثابتة يتم استخدامها للإجابة عن التعبيرات الشرطية، حيث سيتم شرحها بالتفصيل في الفصل الخامس من الحقيبة، ويرمز له بالاختصار (bool)، وتمثل بدالة bool().

مثال: اطبع نتيجة المعادلة (٥ > ٨).

```
print(5>8)
```

النتيجة:

```
False
```

في المثال: تمت طباعة الإجابة (خطأ)، وذلك لعدم صحة التعبير الشرطي،

حيث إن رقم (٥) أصغر من رقم (٨).

مثال: اطبع نتيجة المعادلة (٣ == ٣).

```
print(3==3)
```

النتيجة:

```
True
```

في المثال: تمت طباعة الإجابة (صح)، وذلك لصحة التعبير الشرطي، حيث إن

رقم (٥) يساوي رقم (٥).

مثال: اطبع نتيجة المعادلة (٣ < ٥).

```
print(3<5)
```

النتيجة:

```
True
```

في المثال: تمت طباعة الإجابة (صح)، وذلك لصحة التعبير الشرطي، حيث

إن رقم (٥) أكبر من رقم (٣).

٣. النصوص Strings:

ترمز النصوص في لغة البرمجة للحروف والكلمات والجمل، فهي عبارة عن سلسلة من الحروف أو الكلمات التي ليس لها حجم محدد، وفي لغة بايثون يرمز لها بـ (str)،



وتمثل بدالته (str) ، فجميع أنواع البيانات إذا أضيفت بين علامة التنصيص يتم تحويلها إلى نص

مثال: اكتب برنامج يطبع الحرف (P).

```
print('P')
```

النتيجة:

P

مثال: اكتب برنامج يطبع الكلمة (Python).

```
print('Python')
```

النتيجة:

Python

مثال: اكتب برنامج يطبع الجملة (Hello Python).

```
print('Hello Python')
```

النتيجة:

Hello Python

مثال: اكتب برنامج يطبع الجملة (Hello 2020).

```
print('Hello 2020')
```

النتيجة:

Hello 2020

في المثال: تمت إضافة رقم صحيح مع النص، وتمت طباعته كنص وليس رقماً، وذلك بسبب إضافة الرقم بين علامة التنصيص.

المتغيرات في لغة بايثون Variables :

عبارة عن حاوية يتم إسناد القيمة لها من قبل المبرمج أو المستخدم، ويكون اسمها مختلف عن القيمة المسندة لها، وفي لغة بايثون يتم إسناد القيمة بطريقتين:

- إما مباشرة من قبل المبرمج باستخدام (=)
- أو عن طريقة المستخدم بداله input().

تنبيه: في حالة عدم تحديد نوع المتغير عند الإسناد المباشر باستخدام (=)، فإن بايثون يقوم بتحديد النوع تلقائياً حسب قيمته.



قواعد تسمية المتغير:

في لغة بايثون يوجد عدة شروط لاختيار اسم المتغير:

- يجب ألا يحتوي اسم المتغير على مسافة.
- يمكن استخدام العلامة (_) للفصل بين الكلمات في اسم المتغير.
- يمكن التسمية بالعلامة (_) فقط.
- يمنع استخدام الكلمات المحجوزة في تسمية المتغير مثل: import.
- لغة بايثون تعتبر الحروف الصغيرة والكبيرة حروف مستقلة، ولذلك يمكن استخدام الحرفين A و a، كأسماء لمتغيرين مختلفين.
- اسم المتغير لا بد أن يبدأ بحرف أو (_) ويمكن إضافة أرقام بعدهما مثل: A3، ولا يمكن بداية اسم المتغير برقم.

تعيين قيم المتغيرات في لغة بايثون:

يمكن تعيين قيم المتغيرات في لغة بايثون عن طريقة المبرمج مباشرة من خلال استخدام علامة الإسناد (=) بين القيمة والمتغير، أو من خلال استخدام دالة الإدخال input() لطلب القيمة من المستخدم بشرط تحديد اسم المتغير لها.
أولاً: الإسناد القيم بواسطة (=).

variable	=	value
↑		↑
1		2
		3

١. variable: اسم المتغير

٢. (=): هي رمز الإسناد، بدونها لا يمكن إسناد القيم للمتغير.

٣. value: قيمة المتغير.

طباعة محتوى المتغيرات:

تتم طباعة محتوى المتغيرات من خلال استخدام دالة print() واستدعاء اسم المتغير.

print	(	Variables	)
↑		↑	
1		2	

١. دالة print() للطباعة.

٢. اسم المتغير المطلوب طباعة محتواه.

مثال: اكتب برنامج يطبع قيمة المتغير (x)، حيث قيمة (x) (لغة بايثون سهلة).



```
x='لغة بايثون سهلة'
print(x)
```

النتيجة:

```
لغة بايثون سهلة
```

في المثال: تم تعريف المتغير (x) ثم إسناد القيمة للمتغير، ثم طباعة قيمة المتغير (x).

مثال: اكتب برنامج يطبع قيمة المتغير (x)، حيث قيمته تساوي (٢٠٢٠).

```
x=2020
print(x)
```

النتيجة:

```
2020
```

في المثال: تم تعريف المتغير (x) ثم إسناد القيمة للمتغير، ثم طباعة قيمة المتغير (x).

مثال: اكتب برنامج يطبع قيمة المتغير (x)، حيث قيمة (x) (٢٠٢٠).

```
x=2020
print(x)
```

النتيجة:

```
2020
```

في المثال: تم تعريف المتغير (x) ثم إسناد القيمة للمتغير، ثم طباعة قيمة المتغير (x).

مثال: اكتب برنامج يطبع قيمة المتغير (x)، حيث قيمة (x) (٥<٣).

```
x= 5<3
print(x)
```

النتيجة:

```
False
```

في المثال: تم تعريف المتغير (x) ثم إسناد القيمة للمتغير، ثم طباعة قيمة المتغير (x).

ثانيًا: الإسناد بواسطة المستخدم بدالة input().

دالة input عبارة عن دالة جاهزة في لغة بايثون تقوم بطلب مدخلات من المستخدم وإسنادها

للمتغير المحدد لها.

```
Variable = input(description)
      ↑   ↑   ↑   ↑   ↑
      1   2   3   4   5
```



١. variable: اسم المتغير.
٢. (=): علامة الإسناد.
٣. input: دالة طلب المدخلات.
٤. (): علامة قوس دالة طلب المدخلات.
٥. description: موقع كتابة وصف المدخلات المطلوبة، (جزء اختياري لكتابة وصف عمل الدالة).

مثال: اكتب برنامج يطلب من المستخدم إدخال جملة، وإسنادها للمتغير (x)، وطباعة النتيجة.

```
x = input('Enter Paragraph: ')
print(x)
```

النتيجة:

```
Enter Paragraph: Hello Python
Hello Python
```

في المثال: تم إدخال الجملة (Hello Python) من المستخدم، وتم إسنادها للمتغير x

وطباعة النتيجة.

مثال: اكتب برنامج يطلب من المستخدم إدخال رقم صحيح، ويسنده للمتغير (x)، ثم يطبع

النتيجة.

```
x = input('Enter Int Num: ')
print(x)
```

النتيجة:

```
Enter Int Num: 111
111
```

في المثال: تم إدخال الرقم (١١١) من المستخدم، ثم إسناده للمتغير (x) وطباعة

النتيجة.

مثال: اكتب برنامج يطلب من المستخدم إدخال تعبير مقارنة، وإسناده للمتغير (x)، وطباعة

النتيجة.

```
x = input('Enter Comparison: ')
print(x)
```

النتيجة:

```
Enter Comparison: 5>9
5>9
```



في المثال: تم إدخال تعبير المقارنة ($5 > 9$) من المستخدم، ثم إسناده للمتغير (x) وطباعة النتيجة، لكن لم يتم تنفيذ التعبير الشرطي وإلا لكان الناتج هو (False).

تنويه:

دالة input() تقوم بتحديد نوع المدخلات كنص، ولتفادي هذه المشكلة لابد من تحديد نوع المتغير عند استخدامها.

إسناد أكثر من قيمة بواسطة دالة input() مع دالة split():

split(): عبارة عن دالة جهازه في لغة بايثون تقوم بفصل القيم المسندة للمتغيرات، في هذه الجزئية سوف نستخدمها لفصل القيم المدخلة بواسطة input()، ثم إسنادها لعدة متغيرات.

Variable	=	input	(	Description	)	.	split	(	'Separator'	)
↑		↑		↑		↑		↑		↑
1		2		3		4		5		6
										7
										8
										9

١. variable: اسم المتغير.

٢. (=): علامة الإسناد.

٣. input: دالة طلب المدخلات.

٤. (): علامة قوس دالة طلب المدخلات.

٥. description: موقع كتابة وصف المدخلات المطلوبة، (الدالة تعمل بدون وصف).

٦. (.): نقطة الفصل بين اسم المتغير والدالة.

٧. split: دالة فصل المدخلات.

٨. (): علامة قوس دالة طلب المدخلات.

٩. separator: الفاصل المطلوب استخدامه، عند تركه فارغا سوف يكون الفاصل مسافة، وعند إضافة فاصلة لابد من وضعه بين علامة التخصيص.

مثال: اكتب برنامج يطلب من المستخدم إدخال اسمة الثلاثي بدالة input() واحدة، ثم

يسند كل اسم في متغير، ثم يطبع كل اسم في سطر.

```
a,b,c=input('Enter Full Name with space: ').split()
print(a)
print(b)
print(c)
```



النتيجة:

```
Enter Full Name with space: Thamer Abdullah Alotaibi
Thamer
Abdullah
Alotaibi
```

في المثال: تم تعريف ثلاث متغيرات (a, b, c) في دالة input(), وإضافة دالة split() لفصل المدخلات، ثم إسناد القيم إلى المتغيرات وطباعتها بدالة print(), ملاحظه: لم يتم تحديد نوع الفاصل في دالة split، لذلك تم وضع مسافة بين الأسماء.

تحديد نوع المتغيرات قبل إسنادها:

Variable	=	type	(	input	(	description	)	)
↑		↑		↑		↑		↑
1		2		3		4		5

١. variable: اسم المتغير.
٢. (=): علامة الإسناد.
٣. type: تحديد نوع بيانات المدخلات مثل: int، float.
٤. (:): علامة قوس نوع البيانات.
٥. input: دالة طلب المدخلات من المستخدم.
٦. (:): علامة قوس دالة طلب المدخلات.
٧. description: موقع كتابة وصف المدخلات المطلوبة.

مثال: اكتب برنامج يطلب من المستخدم إدخال رقم صحيح، ويسنده للمتغير (x) ويحدد

نوع المتغير (int)، وطباعة النتيجة.

```
x=int(input('Enter Int: '))
print(x)
```

النتيجة:

```
Enter Int: 300
300
```

في المثال تم إدخال الرقم (٣٠٠) من المستخدم، وتم إسناده للمتغير x وطباعة النتيجة.



```
Enter Int: 10.5
Traceback (most recent call 1
ast):
  File "C:/Users/Tam/AppData/
Local/Programs/Python/Python3
8-32/333.py", line 1, in <mod
ule>
    x=int(input('Enter Int: '
))
ValueError: invalid literal f
or int() with base 10: '10.5'
```

في المثال تم إدخال الرقم (١٠,٥) من المستخدم وظهرت رسالة خطأ تفيد بأن المدخل ليس رقماً صحيحاً.

تنويه: عند تحديد نوع المدخلات في دالة input()، لن تستطيع إدخال أي نوع مختلف عن التحديد.

معرفة أنواع البيانات في لغة بايثون:

في لغة بايثون يمكن عرض نوع البيانات المدخلة من خلال دالة type().

```
type(variable)
↑ ↑ ↑
1 2 3
```

١. type: اسم الدالة.

٢. (): قوس الدالة.

٣. variable: اسم المتغير.

مثال: اكتب برنامج يطبع قيمة المتغير (x)، حيث قيمة (x) (لغة بايثون سهلة)، ثم يطبع نوع المتغير (x).

```
x='لغة بايثون سهلة'
print(x)
print(type(x))
```

النتيجة:

```
لغة بايثون سهلة
<class 'str'>
```

في المثال: تم طباعة نوع المتغير نص ('str').

مثال: اكتب برنامج يطبع قيمة المتغير (x)، حيث قيمة (x) (٢٠٢٠)، ثم يطبع نوع المتغير (x).

```
x=2020
print(x)
print(type(x))
```



النتيجة :

```
2020
<class 'int'>
```

في المثال: تم طباعة نوع المتغير رقم صحيح (class 'int').

مثال: اكتب برنامج يطبع قيمة المتغير (X)، حيث قيمة (X) ($5 < 3$)، ثم يطبع نوع المتغير (X).

```
x=5<3
print(x)
print(type(x))
```

النتيجة :

```
False
<class 'bool'>
```

في المثال: تمت طباعة نوع المتغير البولياني (class 'bool').

مثال: اكتب برنامج يطبع قيمة المتغير (X)، حيث قيمة (X) (3.5)، ثم يطبع نوع المتغير (X).

```
x=3.5
print(x)
print(type(x))
```

النتيجة :

```
3.5
<class 'float'>
```

في المثال: تمت طباعة نوع المتغير رقم عشري (class 'float').

جدول مقارنة أنواع البيانات في لغة بايثون:

البيانات	رمز التصنيف
رقم صحيح	< class 'int'>
رقم عشري	< class 'float'>
قيمة منطقية (البولياني)	< class 'bool'>
نص	< class 'str'>



تخزين النصوص في لغة بايثون:

يتم تخزين النصوص في لغة بايثون في مصفوفة ذات بعد واحد من الحروف، حيث يتم حجز رقم في الذاكرة لكل حرف من حروف النص، وتبدأ المصفوفة من الرقم [٠]، حيث يكون في البرمجة بداية العد من الرقم [٠] وليس الرقم [١]، بناء على ذلك تستطيع طباعة أحرف معينة من النصوص معتمداً على مكانها في المصفوفة.

شرح: يتم تخزين كلمة TVTC في المصفوفة بهذا الشكل:

	T	V	T	C
للطباعة من البداية إلى النهاية	0	1	2	3
للطباعة من النهاية إلى البداية	-4	-3	-2	-1

في المثال كلمة TVTC تبدأ بحرف T ويحتل الموقع [٠]، ثم حرف V يحتل الموقع [١]، ثم T يحتل الموقع [٢]، ثم حرف C يحتل الموقع [٣]. بناء على ذلك يمكنك طباعة أي حرف من النص حسب موقع تخزينه، ويمكن استخدام الأرقام السالبة للطباعة من النهاية إلى البداية، والأرقام الموجبة للطباعة من البداية إلى النهاية.

طباعة حرف معين من النص:

variable[location]		
↑	↑	↑
1	2	3

١. variable: اسم المتغير.

٢. []: قوس المصفوفة

٣. location: رقم الحرف المطلوب في الذاكرة.

مثال: اكتب برنامج يطلب من المستخدم إدخال كلمة ثم يطبعها، ثم يطبع الحرف المتواجد في الخانة الثانية.

```
x=input('Enter Word: ')
print(x)
print(x[2])
```

النتيجة:

```
Enter Word: PYTHON
PYTHON
T
```

في المثال: تم إدخال كلمة (PYTHON)، ثم طباعة الكلمة، ثم طباعة الحرف رقم (2) T.



اقتطاع جزء معين من النص:

يمكن من خلال الاقتطاع طباعة الجزء المقتطع او إسناده لمتغير اخر.

```
variable[first:last]
```

↑ ↑ ↑ ↑ ↑
 1 2 3 4 5

١. variable: اسم المتغير.

٢. []: قوس المصفوفة

٣. first: رقم أول حرف مطلوب اقتطاعه.

٤. (:): نقطتان راسيتان للفصل بين البداية والنهاية.

٥. last: رقم آخر حرف مطلوب اقتطاعه، (الرقم غير مشمول في الاقتطاع).

مثال: اكتب برنامج يطلب من المستخدم إدخال كلمة، ثم يطبعها، ثم يطبع الحرف من ١ إلى ٥.

```
x=input('Enter Word: ')
print(x)
print(x[0:5])
```

النتيجة:

```
Enter Word: Welcome
Welcome
Welco
```

في المثال: تم إدخال الكلمة (welcome)، ثم طباعتها، بعد ذلك طباعة الحروف من الموقع [٠] إلى [٤]، لأن الموقع [٥] لا يكون جزء من الاقتطاع وتكون نتيجة الطباعة كلمة (Welco).
طباعة جزء من نهاية النص يتم استخدام المصفوفة السالبة، حيث يتم إضافة آخر حرف تريد طباعته في خانة (first) وترك خانة (last) فارغة.

مثال: اكتب برنامج يطلب من المستخدم إدخال كلمة، ثم يطبعها، ثم يطبع آخر أربع أحرف.

```
x=input('Enter Word: ')
print(x)
print(x[-4:])
```

النتيجة:

```
Enter Word: Riyadh
Riyadh
yadh
```

في المثال تم إدخال كلمة (Riyadh)، ثم طباعتها، بعد ذلك طباعة آخر أربع حروف (yadh) بواسطة المصفوفة السالبة وذلك لعدم معرفتنا بطول الكلمة المدخلة من المستخدم.



مثال: اكتب برنامج يطلب من المستخدم إدخال أربع قيم، ثم يطبع القيم ونوعها حسب التالي:

- (i): رقم صحيح.
- (f): رقم عشري.
- (b): قيمة منطقية.
- (s): نص.

```
i=input('Enter Integer: ')
f=input('Enter float: ')
b=input('Enter comparison: ')
s=input('Enter String: ')
print(i)
print(type(i))
print(f)
print(type(f))
print(b)
print(type(b))
print(s)
print(type(s))
```

النتيجة:

```
Enter Integer: 5 ← 1
Enter float: 1.5 ← 2
Enter comparison: 5<2 ← 3
Enter String: Apple ← 4
5
<class 'str'>
1.5
<class 'str'>
5<2
<class 'str'>
Apple
<class 'str'>
```

في المثال:

- رقم (١) تم إدخال رقم صحيح (٥)، وتمت طباعة نوع المتغير كنص (str).
 - رقم (٢) تم إدخال رقم عشري رقم (١,٥)، وتمت طباعة نوع المتغير كنص (str).
 - رقم (٣) تم إدخال قيمة منطقية (٥<٢)، وتمت طباعة نوع المتغير كنص (str).
 - رقم (٤) تم إدخال النص (Apple)، وتمت طباعة نوع المتغير كنص (str).
- لذلك عند استخدام دالة الإدخال input()، لابد أن يتم تحديد نوع القيمة المطلوبة ليتم إسنادها للمتغير بالشكل الصحيح، لأن دالة input() تقوم بتحديد جميع المدخلات نصوصاً في حال عدم تحديد نوع القيم المطلوبة.



مثال: أعد تطبيق المثال السابق بعد تحديد أنواع المدخلات المطلوبة من دالة input():

```
i=int(input('Enter Integer: '))
f=float(input('Enter Float: '))
s=str(input('Enter String: '))
b=bool(input('Enter Boolean: '))
print(i)
print(type(i))
print(f)
print(type(f))
print(s)
print(type(s))
print(b)
print(type(b))
```

النتيجة:

```
Enter Integer: 5 ← 1
Enter Float: 1.5 ← 2
Enter String: Apple ← 3
Enter Boolean: 5>8 ← 4
5
<class 'int'>
1.5
<class 'float'>
Apple
<class 'str'>
True
<class 'bool'>
```

في المثال:

- رقم (١) تم إدخال الرقم الصحيح (٥)، ثم طباعة نوع المتغير رقم صحيح (int).
- رقم (٢) تم إدخال الرقم العشري (١,٥)، ثم طباعة نوع المتغير رقم عشري (float).
- رقم (٣) تم إدخال النص (Apple)، ثم طباعة نوع المتغير كنص (str).
- رقم (٤) تم إدخال التعبير البولي (٥>٨)، ثم طباعة نوع المتغير بولي (bool).

بعد تحديد نوع البيانات المطلوب إدخالها من دالة input()، تمت طباعة نوع كل قيمة بشكلها الصحيح.



دوال تتعامل مع النصوص في لغة بايثون :

• دالة strip():

عبارة عن دالة جاهزة تقوم بحذف المسافات الموجودة في بداية ونهاية القيم المدخلة.

```
variable.strip()
↑   ↑   ↑   ↑
1   2   3   4
```

١. variable : اسم المتغير.

٢. (.): نقطة الفصل بين اسم المتغير والدالة.

٣. strip : اسم الدالة.

٤. (): قوس الدالة.

مثال:

اكتب برنامج يطلب من المستخدم إدخال اسمه وإضافة مسافة قبل وبعد الاسم، ثم يطبع الاسم، ثم يعيد طباعة الاسم مع حذف المسافات.

```
x=input('Enter Name with space: ')
print(x)
print(x.strip())
```

النتيجة:

```
Enter Name with Space: Azaam
Azaam | ← 1
Azaam ← 2
```

في النتيجة رقم (1) تمت طباعة الاسم المدخل مع المسافات المدخلة من قبل المستخدم، أما في النتيجة (2) فتمت إزالة المسافات التي تم إدخالها من قبل المستخدم قبل بداية الاسم و بعد نهايته، وذلك لاستخدام دالة strip().

• دالة capitalize():

عبارة عن دالة جاهزة تقوم بتحويل الحرف الأول إلى حرف كبير. هذه الدالة تعمل مع اللغة الإنجليزية.

```
variable.capitalize()
↑   ↑   ↑   ↑
1   2   3   4
```



١. variable : اسم المتغير.

٢. (.): نقطة الفصل بين اسم المتغير والدالة.

٣. capitalize : اسم الدالة.

٤. (): قوس الدالة.

مثال:

اكتب برنامج يطلب من المستخدم إدخال اسمه، ثم يطبع الاسم، ثم يعيد طباعة الاسم بعد تحويل الحرف الأول إلى حرف كبير.

```
x=input('Enter Name: ')
print(x)
print(x.capitalize())
```

النتيجة:

```
Enter Name: google
google ← 1
Google ← 2
```

في النتيجة رقم (1) تمت طباعة الاسم بنفس طريقة الإدخال حيث إن الحرف الأول صغير، أما في النتيجة (2) تمت طباعة الاسم بعد تحويل الحرف الأول إلى حرف كبير وذلك لاستخدام دالة capitalize().

• دالة upper():

عبارة عن دالة جاهزة تقوم بتحويل حروف النص المدخل من حروف صغيرة إلى كبيرة.

```
variable.upper()
↑   ↑   ↑   ↑
1   2   3   4
```

١. variable : اسم المتغير.

٢. (.): نقطة الفصل بين اسم المتغير والدالة.

٣. upper : اسم الدالة.

٤. (): قوس الدالة.

مثال:

اكتب برنامج يطلب من المستخدم إدخال نص، ثم يطبع النص، ثم يعيد طباعة النص بعد تحويل جميع الحروف إلى حروف كبيرة.



```
x=input('Enter Text: ')
print(x)
print(x.upper())
```

النتيجة:

```
Enter Text: google
google ← 1
GOOGLE ← 2
```

في النتيجة رقم (1) تمت طباعة النص بنفس طريقة الإدخال حيث إن الحروف صغيرة، أما في النتيجة (2) تمت طباعة النص بعد تحويل حروفه إلى حروف كبيرة وذلك لاستخدام دالة `upper()`.

• دالة `lower()`:

عبارة عن دالة جاهزة تقوم بتحويل حروف النص المدخل من حروف كبيرة إلى صغيرة.

```
variable.lower()
```

↑ ↑ ↑ ↑
1 2 3 4

١. `variable`: اسم المتغير.

٢. `(.)`: نقطة الفصل بين اسم المتغير والدالة.

٣. `lower`: اسم الدالة.

٤. `( )`: قوس الدالة.

مثال:

اكتب برنامج يطلب من المستخدم إدخال نص بحروف كبيرة، ثم يطبع النص، ثم يعيد طباعة النص بعد تحويل جميع الحروف إلى حروف صغيرة.

```
x=input('Enter Text: ')
print(x)
print(x.lower())
```

النتيجة:

```
Enter Text: GOOGLE
GOOGLE ← 1
google ← 2
```



في النتيجة رقم (1) تمت طباعة النص بنفس طريقة الإدخال حيث إن الحروف كبيرة، أما في النتيجة (2) تمت طباعة النص بعد تحويل حروفه إلى حروف صغيرة وذلك لاستخدام دالة `lower()`.

• دالة `title()`:

عبارة عن دالة جاهزة تقوم بتحويل أول حرف من كل كلمة إلى حرف كبير، حيث تستخدم غالباً في العناوين.

```
variable.title()
  ↑   ↑   ↑   ↑
  1   2   3   4
```

١. `variable`: اسم المتغير.

٢. `(.)`: نقطة الفصل بين اسم المتغير والدالة.

٣. `title`: اسم الدالة.

٤. `( )`: قوس الدالة.

مثال:

اكتب برنامج يطلب من المستخدم إدخال نص، ثم يطبع النص، ثم يعيد طباعة النص بعد تحويل أول حرف من كل كلمة إلى حرف كبير.

```
x=input('Enter Text: ')
print(x)
print(x.title())
```

النتيجة:

```
Enter Text: time to learn python
time to learn python ← 1
Time To Learn Python ← 2
```

في النتيجة رقم (1) تمت طباعة النص بنفس طريقة الإدخال حيث إن أول حرف من كل كلمة صغير، أما في النتيجة (2) تمت طباعة النص بعد تحويل أول حرف من كل كلمة إلى حروف كبير وذلك لاستخدام دالة `title()`.



• دالة replace():

عبارة عن دالة جاهزة تقوم باستبدال الحروف أو النصوص المسندة لمتغير.

```
variable.replace(old, new)
```

↑ ↑ ↑ ↑ ↑ ↑
 1 2 3 4 5 6 7

١. variable: اسم المتغير.

٢. (.): نقطة الفصل بين اسم المتغير والدالة.

٣. replace: اسم الدالة.

٤. (): قوس الدالة.

٥. old: الحرف أو النص المطلوب استبداله.

٦. (,): فاصلة الفصل بين القيمة القديمة والجديدة.

٧. new: الحرف أو النص الجديد.

ملاحظته: دالة replace() حساسة لحالة الحروف، يجب كتابة النص بنفس التنسيق.

مثال:

اكتب برنامج يطلب من المستخدم طباعة النص (Time To learn Python)، ثم

يقوم باستبدال كلمة (learn) بـ (Use) ثم طباعة النص.

```
x='Time To Learn Python'
print(x)
print(x.replace('Learn','Use'))
```

النتيجة:

```
Time To Learn Python ← 1
Time To Use Python ← 2
```

في النتيجة رقم (1) تمت طباعة النص بنفس طريقة الإسناد، أما في النتيجة (2)

تمت طباعة النص بعد استبدال كلمة (Learn) بكلمة (Use)، وذلك لاستخدام دالة

replace()



تحويل نوع المدخلات بواسطة eval():

الاستخدام الثاني لدالة eval() تحويل المدخلات إلى نوع البيانات الصحيح.

مثال: اكتب برنامج يطلب من المستخدم إدخال أربع قيم، ثم يطبع القيم ونوعها حسب التالي:

- (i): رقم صحيح.
- (f): رقم عشري.
- (b): قيمة منطقية.

```
i=eval(input('Enter Int: '))
f=eval(input('Enter Float: '))
b=eval(input('Enter Bool: '))
print(i)
print(type(i))
print(f)
print(type(f))
print(b)
print(type(b))
```

النتيجة:

```
Enter Int: 4
Enter Float: 5.5
Enter Bool: 5<4
4
<class 'int'> ← 1
5.5
<class 'float'> ← 2
False
<class 'bool'> ← 3
```

في المثال:

- رقم (١) تم إدخال الرقم الصحيح (٤)، ثم طباعة نوع المتغير رقم صحيح (int).
 - رقم (٢) تم إدخال الرقم العشري بقيمة (٥,٥)، ثم طباعة نوع المتغير رقم عشري (float).
 - رقم (٣) تم إدخال القيمة المنطقية (٥<٤)، ثم طباعة نوع المتغير البولياني (bool).
- عند استخدام دالة eval() مع دالة input() فإنها تقوم تلقائياً بتحديد نوع المدخلات حسب تصنيفها في لغة بايثون. تم استبعاد إدخال النصوص وذلك لأن دالة input() تقوم بتحديد نوع المدخلات نصوصاً.



اكتب برنامج يطلب من المستخدم إدخال كلمة True أو False، ثم يطبع نوع المتغير.

```
x=eval(input('Choose True or False: '))
print(x)
print(type(x))
```

النتيجة:

```
Choose True or False: False
False
<class 'bool'>
```

في المثال طلب من المستخدم الاختيار بين True أو False، حيث تم اختيار (False)، ثم إسنادها للمتغير (x) ثم طباعة نوع المتغير.



تمارين الوحدة

١. اذكر ثلاثة من أنواع البيانات في لغة بايثون.
٢. اذكر الفرق بين الأرقام الصحيحة والعشرية في لغة بايثون.
٣. اذكر ثلاثة من قواعد تسمية المتغير.
٤. اذكر طرق إدخال القيم للمتغير مع ذكر مثال لكل نوع.
٥. اكتب البرامج التالية مع رسم مخطط انسيابي لكل برنامج:
 - برنامج يطلب من المستخدم إدخال أرقام صحيحة وعشرية فقط ويسندها لمتغير ثم يطبع المتغير ونوع قيمة المتغير.
 - برنامج يطلب من المستخدم إدخال أرقام صحيحة فقط ويسندها لمتغير ثم يطبع المتغير ونوع قيمة المتغير.
 - اكتب برنامج يطبع اسمك، عمرك، معدلك، بشرط تحديد النوع الصحيح لكل متغير.
 - اكتب برنامج يطلب من المستخدم الاسم، الطول، الوقت، ويسندها لثلاث متغيرات الاسم str، الطول int، الوقت float، ثم يقوم بطباعة قيم المتغيرات على الشاشة مع النوع.
 - اكتب برنامج يطلب من المستخدم تاريخ اليوم ويكون بالصيغة التالية: yyyy.mm.dd ويسند للمتغير Date ويتم تحديد نوع البيانات المدخلة تلقائياً.
 - اكتب برنامج يقوم بتوضيح الوظيفة الأساسية لدالة eval().
 - اكتب برنامج يطلب من المستخدم الاسم والعمر والطول بمدخل واحد فقط ويقوم بإسناد البيانات لثلاثة متغيرات مختلفة ثم طباعة كل متغير في سطر مستقل.



نموذج تقييم المتدرب لمستوى أدائه					
يعبأ من قبل المتدرب نفسه وذلك بعد الانتهاء من تمارين الوحدة					
بعد الانتهاء من التدريب على وحدة أنواع البيانات والمتغيرات، قيم نفسك وقدراتك بواسطة إكمال هذا التقييم الذاتي بعد كل عنصر من العناصر المذكورة، وذلك بوضع علامة (✓) أمام مستوى الأداء الذي أتقنته، وفي حالة عدم قابلية المهمة للتطبيق ضع العلامة في الخانة الخاصة بذلك.					
م	العناصر	مستوى الأداء (هل أتقنت الأداء)			
		غير قابل للتطبيق	لا	جزئيا	كليا
١	يوضح أنواع البيانات في لغة بايثون.				
٢	يستخدم الدوال في لغة بايثون.				
٣	يمثل المتغيرات في لغة بايثون.				
يجب أن تصل النتيجة لجميع المفردات (البُود) المذكورة إلى درجة الإتقان الكلي أو غير قابلة للتطبيق، وفي حالة وجود مفردة في القائمة "لا" أو "جزئيا" فيجب إعادة التدريب على هذا النشاط مرة أخرى بمساعدة المدرب.					



نموذج تقييم المدرب لمستوى أداء المتدرب					
يعبأ من قبل المدرب وذلك بعد الانتهاء من تمارين الوحدة					
اسم المتدرب :		التاريخ:			
رقم المتدرب :		المحاولة: ١ ٢ ٣ ٤			
		العلامة:			
كل بند أو مفردة يقيم بـ ١٠ نقاط الحد الأدنى: ما يعادل ٨٠٪ من مجموع النقاط. الحد الأعلى: ما يعادل ١٠٠٪ من مجموع النقاط.					
م	بنود التقييم	النقاط (حسب رقم المحاولات)			
		١	٢	٣	٤
١	يوضح أنواع البيانات في لغة بايثون.				
٢	يستخدم الدوال في لغة بايثون.				
٣	يمثل المتغيرات في لغة بايثون.				
المجموع					
ملحوظات:					
.....					
توقيع المدرب:					

الوحدة الرابعة

العمليات الحسابية



الوحدة الرابعة

العمليات الحسابية

الهدف العام للوحدة:

تهدف هذه الوحدة إلى إتقان المتدرب العمليات الحسابية واستخدام بعض الدوال الرياضية في لغة بايثون.

الأهداف التفصيلية:

من المتوقع في نهاية هذه الوحدة التدريبية أن يكون المتدرب قادراً وبكفاءة على أن:

١. ينفذ التعبيرات الرياضية في لغة بايثون.
٢. يستعمل الدوال الرياضية في لغة بايثون.
٣. يستدعي المكتبة مع دوالها في لغة بايثون.
٤. يطبق معاملات الإسناد الرياضية في لغة بايثون.

الوقت المتوقع للتدريب على هذه الوحدة: ١٥ ساعة تدريبية.

الوسائل المساعدة:

١. جهاز حاسب آلي.
٢. جهاز عرض (Data Show).
٣. محرر كتابة Python.



المعاملات الحسابية:

تستطيع لغة بايثون حل العمليات الرياضية وتستخدم الترتيب المتفق عليه من قبل الرياضيين لحل تكل العمليات مع اختلاف بسيط في استخدام بعض الرموز وذلك ما سنوضحه في هذه الوحدة

يتم حل العمليات الحسابية في لغة بايثون بطريقتين:

- عن طريق Shell مباشرة.
- عن طريق كتابة الكود البرمجي بالمتغيرات والشروط أو استخدام دالة eval() كما تعلمنا في الوحدة السابقة.

تنبيه: يجب تحديد نوع البيانات إذا ما كانت أرقام صحيحة أو عشرية ليتم التعامل مع المعاملات الحسابية بشكل صحيح في الكود البرمجي وذلك من خلال استخدام التحويل بواسطة دالة eval() أو تحديد نوع المدخل من دالة input().

رموز المعاملات الحسابية:

الرمز	الوظيفة	مثال حسابي
الجمع (+)	يقوم بجمع العناصر	$3+2=5$
الطرح (-)	يقوم بطرح العناصر	$3-5=2$
الضرب (*)	يقوم بضرب العناصر	$2*3=6$
القسمة (/)	يقوم بقسمة العناصر	$3/6=2$

كيفية تعامل لغة بايثون مع المعاملات الحسابية:

تتعامل لغة بايثون مع العمليات الحسابية في Shell مباشرة مثل التعامل مع آلة الحاسبة، حيث تحتاج لكتابة الأرقام والمعاملات الحسابية لتنفيذ العمليات.



١. الجمع (+):

- طريقة تعامل لغة بايثون مع الجمع في Shell:

```
>>> 3+2
5
>>> 2+2
4
>>>
```

- طريقة تعامل لغة بايثون مع الجمع في البرمجة.

مثال:

اكتب برنامج يطلب رقمين من المستخدم ويطبع حاصل جمعهما ، حيث الرقم

الأول صحيح والثاني عشري.

```
1→ N1=int(input('1st Num(Int): '))
2→ N2=float(input('2nd Num(Float): '))
3→ x=N1+N2
4→ print(x)
5→ print(N1+N2)
```

١. المتغير الأول وتم تحديده كرقم صحيح.

٢. المتغير الثاني وتم تحديده كرقم عشري.

٣. متغير X لجمع المتغير الأول والثاني.

٤. طباعة قيمة المتغير X.

٥. تنفيذ عملة الجمع في دالة print() وطباعة النتيجة مباشرة.

النتيجة:

```
1st Num(Int): 5
2nd Num(Float): 2.5
7.5 ← 1
7.5 ← 2
>>>
```

كما تلاحظ تم حل المثال بطريقتين:

- الحل الأول بتعريف متغير تم إسناد حاصل عملية الجمع ، ثم طباعة قيمة المتغير.
- الحل الثاني تنفيذ عملية الجمع مباشرة في دالة print() وطباعة النتيجة.



حل المثال أعلاه باستخدام دالة eval():

```
x=eval(input('اجمع عدد صحيح مع عشري: '))
print(x)
```

النتيجة:

```
5+2.5 : اجمع عدد صحيح مع عشري
7.5
>>>
```

عند استخدام دالة eval() تم اختصار الكود البرمجي لسطرين فقط.

٢. الطرح (-):

• طريقة تعامل لغة بايثون مع الطرح في Shell:

```
>>> 3-5
-2
>>> 2.-1.5
0.5
>>>
```

• طريقة تعامل لغة بايثون مع الطرح في البرمجة.

مثال:

اكتب برنامج يطلب رقمين من المستخدم ويطبّع حاصل طرحهما، حيث

الرقم الأول صحيح والثاني عشري.

```
1→x=int(input('1st Num(Int): '))
2→y=float(input('2nd Num(Float): '))
3→r=x-y
4→print(r)
5→print(x-y)
```

١. المتغير الأول وتم تحديده كرقم صحيح.

٢. المتغير الثاني وتم تحديده كرقم عشري.

٣. متغير x لطرح المتغير الأول والثاني.

٤. طباعة قيمة المتغير X.

٥. تنفيذ عملة الطرح في دالة print() وطباعة النتيجة.

النتيجة:

```
1st Num(Int): 5
2nd Num(Float): 2.5
2.5 ← 1
2.5 ← 2
```



كما تلاحظ تم حل المثال بطريقتين :

الحل الأول بتعريف متغير تم إسناد حاصل نتيجة الطرح، ثم طباعة قيمة المتغير.

الحل الثاني تنفيذ عملية الطرح مباشرة في دالة print() وطباعة النتيجة.

حل المثال أعلاه باستخدام دالة eval():

```
x=eval(input('اطرح عدد صحيح مع عشري: '))
print(x)
```

النتيجة:

```
5-2.5 : ا طرح عدد صحيح مع عشري
2.5
>>>
```

٣. الضرب (*):

• طريقة تعامل لغة بايثون مع الضرب في Shell:

```
>>> 5*5
25
>>> 2.5*2.5
6.25
>>>
```

• طريقة تعامل لغة بايثون مع الضرب في البرمجة.

مثال:

اكتب برنامج يطلب رقمين من المستخدم ويطبع حاصل ضربيهما، حيث

الرقم الأول صحيح والثاني عشري.

```
1→ N1=int(input('1st Num(Int): '))
2→ N2=float(input('2nd Num(Float): '))
3→ x=N1*N2
4→ print(x)
5→ print(N1*N2)
```

١. المتغير الأول وتم تحديده كرقم صحيح.

٢. المتغير الثاني وتم تحديده كرقم عشري.

٣. متغير x لضرب المتغير الأول والثاني.

٤. طباعة قيمة المتغير x.

٥. تنفيذ عملة الضرب في دالة print() وطباعة النتيجة.



النتيجة:

```
1st Num(Int): 3
2nd Num(Float): 2.5
7.5 ← 1
7.5 ← 2
```

كما تلاحظ تم حل المثال بطريقتين :

الحل الأول بتعريف متغير تم إسناد حاصل نتيجة الضرب، ثم طباعة قيمة المتغير.

الحل الثاني تنفيذ عملية الضرب مباشرة في دالة print() وطباعة النتيجة.

حل المثال أعلاه باستخدام دالة eval():

```
x=eval(input('عدد صحيح مع عشري اضرب: '))
print(x)
```

النتيجة:

```
اضرب عدد صحيح مع عشري : 3*1.5
4.5
>>>
```

٤. القسمة (/):

• طريقة تعامل لغة بايثون مع القسمة في Shell:

```
>>> 6/2
3.0
>>> 3/3
1.0
```

• طريقة تعامل لغة بايثون مع القسمة في البرمجة.

مثال:

اكتب برنامج يطلب رقمين من المستخدم ويطبع حاصل قسمتهما، حيث

الرقم الأول صحيح والثاني عشري.

```
1→N1=int(input('1st Num(Int): '))
2→N2=float(input('2nd Num(Float): '))
3→x=N1/N2
4→print(x)
5→print(N1/N2)
```

١. المتغير الأول وتم تحديده كرقم صحيح.

٢. المتغير الثاني وتم تحديده كرقم عشري.



٣. متغير X لقسمة المتغير الأول والثاني.

٤. طباعة قيمة المتغير X.

٥. تنفيذ عملة القسمة في دالة print() وطباعة النتيجة.

النتيجة:

```
1st Num(Int): 6
2nd Num(Float): 1.5
4.0 ← 1
4.0 ← 2
```

كما تلاحظ تم حل المثال بطريقتين :

الحل الأول بتعريف متغير تم إسناد حاصل نتيجة القسمة ، ثم طباعة قيمة المتغير.

الحل الثاني تنفيذ عملية القسمة مباشرة في دالة print() وطباعة النتيجة.

حل المثال أعلاه باستخدام دالة eval():

```
x=eval(input('اقسم عدد صحيح مع عشري: '))
print(x)
```

النتيجة:

```
اقسم عدد صحيح مع عشري: 8/1.7
4.705882352941177
>>>
```

٥. باقي القسمة (%):

يعرف باقي القسمة بأنه العدد الباقي من عملية القسمة ، مثال: قسمة الرقم

(5) على الرقم (2) يكون باقي القسمة (١). في لغة بايثون يستخدم الرمز (%)

للاستدلال على باقي القسمة.

• طريقة تعامل لغة بايثون مع باقي القسمة في Shell:

```
>>> 5%2
1
>>> 5%1.5
0.5
>>>
```

• طريقة تعامل لغة بايثون مع باقي القسمة في البرمجة.



مثال:

اكتب برنامج يطلب رقمين من المستخدم ويطبّع باقي قسمتهما، حيث الرقم

الأول صحيح والثاني عشري.

```
1→N1=int(input('1st Num(Int): '))
2→N2=float(input('2nd Num(Float): '))
3→x=N1%N2
4→print(x)
5→print(N1%N2)
```

١. المتغير الأول وتم تحديده كرقم صحيح.

٢. المتغير الثاني وتم تحديده كرقم عشري.

٣. متغير x لحساب ناتج قسمة الرقم الأول على الرقم الثاني.

٤. طباعة قيمة المتغير x.

٥. حساب ناتج القسمة في دالة print() وطباعة النتيجة.

النتيجة:

```
1st Num: 5
2nd Num: 1.5
0.5 ← 1
0.5 ← 2
>>>
```

في المثال تم الحل بطريقتين :

الحل الأول بتعريف متغير ثم إسناد ناتج القسمة له، ثم طباعة قيمة المتغير.

الحل الثاني حساب ناتج القسمة في دالة print() وطباعة النتيجة.

حل المثال أعلاه باستخدام دالة eval():

```
x=eval(input('القسمة لعدد صحيح وعشري باقي احسب: '))
print(x)
```

النتيجة:

```
5%1.5 : احسب باقي القسمة لعدد صحيح وعشري
0.5
>>>
```

٦. معامل الأس (**):

يعرف معامل الأس بأنه عدد مرات ضرب العدد في نفسه، مثال: ضرب الرقم

(2) ثلاث مرات في نفسه ويكتب في الرياضيات بهذا الشكل (2^3)، وفي لغة بايثون

يكتب بهذا الشكل ($2^{**}3$)، حيث الرمز (**) للاستدلال على معامل الأس.



- طريقة تعامل لغة بايثون مع معامل الأس في Shell:

```
>>> 2**3
8
>>> 3**2
9
>>>
```

- طريقة تعامل لغة بايثون مع معامل الأس في البرمجة.

مثال:

اكتب برنامج يطلب رقم ومعامل الأس من المستخدم ويطبع النتيجة.

```
1→N1=float(input('1st Num: '))
2→N2=float(input('2nd Num(Expo): '))
3→x=N1**N2
4→print(x)
5→print(N1**N2)
```

١. المتغير الأول وتم تحديده كرقم عشري لإسناد الرقم.
٢. المتغير الثاني وتم تحديده كرقم عشري لإسناد الأس.
٣. متغير X لحساب نتيجة عملية الأس.
٤. طباعة قيمة المتغير X.
٥. حساب ناتج عملية الأس في دالة print() وطباعة النتيجة.

النتيجة:

```
1st Num(Int): 2
2nd Num(Expo): 3
8.0 ← 1
8.0 ← 2
```

في المثال تم الحل بطريقتين :

الحل الأول بتعريف متغير ثم إسناد ناتج عملية الأس، ثم طباعة قيمة المتغير.

الحل الثاني تم حساب ناتج عملية الأس في دالة print() وطباعة النتيجة.

حل المثال أعلاه باستخدام دالة eval():

```
x=eval(input('ادخل معادلة الأس: '))
print(x)
```

النتيجة:

```
ادخل معادلة الأس: 3**3
27
>>>
```




أولويات المعاملات الحسابية:

هي ترتيب تنفيذ العمليات الرياضية في المعادلات، كما تعلم في الرياضيات هناك ترتيب لتنفيذ العمليات، أيضاً لغة بايثون تقوم بتنفيذ العمليات الرياضية وفق ترتيب وتسلسل معين.

ترتيب العمليات في لغة بايثون:

الترتيب	العملية
1	العمليات داخل الأقواس
2	رفع الأس
3	الضرب والقسمة
4	الجمع والطرح

مثال:

قم بحساب ناتج المعادلة التالية: $3 + 4 * 4 + 5 * (4 + 3) - 1$

الحل:

١. تنفيذ العملية داخل الأقواس: $7 = (4+3)$

٢. تنفيذ عمليات الضرب: $16 = (4 * 4)$

٣. تنفيذ عملية الضرب: $35 = (5 * 7)$

٤. تنفيذ عملية الجمع: $19 = (3 + 16)$

٥. تنفيذ عملية الجمع: $54 = (19 + 35)$

٦. تنفيذ عملية الطرح: $54 - 1$

٧. الناتج: 53



شرح الحل:

$$\begin{array}{lcl}
 3 + 4 * 4 + 5 * (4 + 3) - 1 & & \\
 \uparrow & \text{(1) العملية داخل الأقواس} & \\
 3 + 4 * 4 + 5 * 7 - 1 & & \\
 \uparrow & \text{(2) عملية الضرب} & \\
 3 + 16 + 5 * 7 - 1 & & \\
 \uparrow & \text{(3) عملية الضرب} & \\
 3 + 16 + 35 - 1 & & \\
 \uparrow & \text{(4) عملية الجمع} & \\
 19 + 35 - 1 & & \\
 \uparrow & \text{(5) عملية الجمع} & \\
 54 - 1 & & \\
 \uparrow & \text{(6) عملية الطرح} & \\
 \boxed{53} & &
 \end{array}$$

كتابة التعبيرات الحسابية

يقصد بالتعبيرات الحسابية هنا المعادلات الرياضية، في بايثون يتم دعم كتابة المعادلات الرياضية وتنفيذها وذلك من خلال تحويل وترتيب العمليات.

مثال:

$$\frac{3 + 4x}{5} - \frac{10(y - 5)(a + b + c)}{x} + 9\left(\frac{4}{x} + \frac{9 + x}{y}\right)$$

الحل:

$(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x + 9 * (4 / x + (9 + x) / y)$
 كما تلاحظ تم تحويل الكسور إلى أقواس وكتابة المقام والبسط بتعبير رياضي مبسط مبني على رموز العمليات الرياضية في بايثون.

مثال:

$$\frac{4}{3(r + 34)} - 9(a + bc) + \frac{3 + d(2 + a)}{a + bd}$$

الحل:

$$(4) / 3 * (r + 34) - 9 * (a + b * c) + (3 + d * (2 + a)) / (a + b * d)$$

دوال رياضية جاهزة في لغة بايثون:

يوجد في لغة بايثون مجموعة من الدوال الرياضية الجاهزة التي تساعد في تنفيذ العمليات بسهولة، حيث تنقسم إلى قسمين: القسم الأول يعمل مباشرة، القسم الثاني يحتاج إلى استدعاء مكتبة المعادلات الرياضية (math)، في هذه الحقيبة سوف نتعلم مجموعة منها وهي:



١. دالة max():

عبارة عن دالة تقوم باختيار القيمة الكبرى لمجموعة من القيم، سواء نصية أو رقمية.

```
max(n1, n2, n3, ..., n6)
```

↑ ↑ ↑ ↑ ↑ ↑
1 2 3 4 5 6

١. يتم كتابة max.

٢. فتح القوس.

٣. وضع القيمة الأولى أو المتغير.

٤. وضع علامة فاصلة.

٥. إضافة القيمة الثانية أو المتغير (الاستمرار بوضع الفاصلة والقيمة حسب المطلوب).

٦. إغلاق القوس.

مثال:

اكتب برنامج يطلب من المستخدم ثلاث أرقام مختلفة، ثم يطبع القيمة الأكبر.

```
a=float(input('1st Num: '))
b=float(input('2nd Num: '))
c=float(input('3rd Num: '))
d=max(a,b,c)
print(d)
print(max(a,b,c))
```

النتيجة:

```
1st Num: 4
2nd Num: 7
3rd Num: 9
9.0 ← 1
9.0 ← 2
```

كما تلاحظ يمكن استخدام دالة max() بطريقتين:

الأولى تعريف متغير وإضافة الدالة له ثم طباعة قيمة المتغير.

الثانية إضافة الدالة مع دالة print() وتنفيذ المقارنة من خلالها.

تنويه:

دالة max() تعمل مع النصوص في بايثون حيث تعتمد على الترتيب الهجائي

عند اختيار الكلمة، حيث تطبع الكلمة التي تبدأ بأخر حرف.



مثال:

اكتب برنامج يطلب من المستخدم ثلاث كلمات مختلفة، ثم يطبع الكلمة التي تبدأ بأخر حرف.

```
a=input('1st Word: ')
b=input('2nd Word: ')
c=input('3rd: Word ')
d=max(a,b,c)
print(d)
print(max(a,b,c))
```

النتيجة:

```
1st Word: Swift
2nd Word: Matlab
3rd: Word Python
Swift ← 1
Swift ← 2
```

كما تلاحظ تمت طباعة الكلمة التي تبدأ بالحرف ذو الترتيب الهجائي الأخير.

٢. دالة min():

عبارة عن دالة تقوم باختيار القسمة الصغرى لمجموعة من القيم، سواء نصية أو رقمية، وهي عكس دالة max().

```
min(n1,n2,n3....)
↑ ↑ ↑ ↑ ↑
1 2 3 4 5 6
```

١. يتم كتابة min.

٢. فتح القوس.

٣. وضع القيمة الأولى أو المتغير.

٤. وضع علامة فاصلة.

٥. إضافة القيمة الثانية أو المتغير (الاستمرار بوضع الفاصلة والقيمة حسب المطلوب).

٦. إغلاق القوس.



مثال:

اكتب برنامج يطلب من المستخدم ثلاث أرقام مختلفة، ثم يطبع القيمة

الأصغر.

```
a=float(input('1st Num: '))
b=float(input('2nd Num: '))
c=float(input('3rd Num: '))
d=min(a,b,c)
print(d)
print(min(a,b,c))
```

النتيجة:

```
1st Num: 8
2nd Num: 5
3rd Num: 2
2.0 ← 1
2.0 ← 2
```

كما تلاحظ يمكن استخدام دالة min() بطريقتين:
الأولى تعريف متغير وإضافة الدالة له ثم طباعة قيمة المتغير.
الثانية إضافة الدالة مع دالة print() وتنفيذ المقارنة من خلالها.

تنويه:

دالة min() تعمل مع النصوص في بايثون حيث تعتمد على الترتيب الهجائي عند اختيار الكلمة، حيث تطبع الكلمة التي تبدأ بأول حرف.

مثال:

اكتب برنامج يطلب من المستخدم ثلاث كلمات مختلفة، ثم يطبع الكلمة

التي تبدأ بأول حرف

```
a=input('1st Word: ')
b=input('2nd Word: ')
c=input('3rd Word: ')
d=min(a,b,c)
print(d)
print(min(a,b,c))
```

النتيجة:

```
1st Word: Swift
2nd Word: Matlab
3rd Word: Python
Matlab ← 1
Matlab ← 2
```

كما تلاحظ تمت طباعة الكلمة التي تبدأ بأول حرف.



دالة pow():

عبارة عن دالة تقوم بحساب نتيجة رفع الأساس إلى قوة الأس، حيث تقوم بتمرير العدد والأس ثم تقوم بحساب الناتج، وهي شبيهة بـ (**).

```
pow ( x , z )
↑   ↑   ↑   ↑   ↑   ↑
1   2   3   4   5   6
```

١. كتابة اسم الدالة pow.
٢. فتح القوس بعد اسم الدالة.
٣. (x) مكان إضافة الرقم.
٤. وضع الفاصلة.
٥. (z) مكان إضافة رقم الأس.
٦. إغلاق القوس بعد رقم الأس.

مثال:

اكتب برنامج يطلب من المستخدم رقم والأس ثم طباعة الناتج.

```
a=float(input('The Num: '))
b=float(input('The Power: '))
c=pow(a,b)
print(c)
print(pow(a,b))
```

النتيجة:

```
The Num: 3
The Power: 2
9.0 ← 1
9.0 ← 2
```

كما تلاحظ تم إسناد الرقم و الأس ثم حل الحل بطريقتين:
الأولى بتعريف متغير ثم إضافة دالة pow() له ثم طباعة قيمة المتغير،
الثانية إضافة دالة pow() مع دالة print() وطباعة النتيجة.

٤. دالة round():

عبارة عن دالة تقوم بتقريب العدد العشري إلى أقرب عدد صحيح أو إلى أقرب خانة عشرية مطلوبة.



يمكن استخدام دالة round() بطريقتين:

- التقريب إلى أقرب عدد صحيح.

```
round ( x )
↑   ↑ ↑ ↑
1   2 3 4
```

١. كتابة اسم الدالة round.

٢. فتح علامة القوس مباشرة بعد اسم الدالة.

٣. كتابة الرقم أو المتغير المطلوب تقريبه.

٤. إغلاق القوس بعد الرقم.

مثال:

اكتب برنامج يطلب من المستخدم رقم عشري، ثم يقربه إلى أقرب رقم

صحيح.

```
x=float(input('Enter Num: '))
r=round(x)
print(r)
print(round(x))
```

النتيجة:

```
Enter Num: 3.6
4 ← 1
4 ← 2
```

كما تلاحظ تم التعامل مع دالة round() بطريقتين مختلفتين:

الأولى تم تعريف متغير x وإضافة دالة round() مع الرقم المدخل، ثم

طباعة المتغير x،

الثانية تم إضافة دالة round() مع دالة print() وطباعة الرقم بعد التقريب.

- التقريب إلى أقرب عدد عشري.

```
round ( x , z )
↑   ↑ ↑ ↑ ↑ ↑
1   2 3 4 5 6
```

١. كتابة اسم الدالة round.

٢. فتح علامة القوس مباشرة بعد اسم الدالة.

٣. كتابة الرقم أو المتغير المطلوب تقريبه.



٤. إضافة فاصلة.

٥. كتابة عدد الأرقام العشرية المطلوب التقريب لها ، علماً بأن الرقم يجب أن يكون رقماً صحيحاً وليس عشرياً.

٦. إغلاق القوس بعد الرقم.

مثال:

اكتب برنامج يطلب من المستخدم رقم عشري ، ثم يطبع الرقم بعد تقريبه

إلى أقرب خانتين عشريتين.

```
x=float(input('Enter Num: '))
r=round(x,2)
print(r)
print(round(x,2))
```

النتيجة:

```
Enter Num: 3.14444
3.14 ← 1
3.14 ← 2
```

كما تلاحظ تم التعامل مع دالة round() بطريقتين مختلفتين:

الأولى تم تعريف متغير x وإضافة دالة round() مع الرقم المدخل و تحديد

عدد الخانات العشرية ثم طباعة المتغير x،

الثانية تم إضافة دالة round() مع دالة print() مع تحديد عدد الخانات

العشرية وطباعة الرقم بعد التقريب.

٥. دالة sqrt():

عبارة عن دالة تقوم بحساب الجذر التربيعي للرقم.

```
sqrt( x )
↑ ↑ ↑
1 2 3
```

١. كتابة اسم الدالة sqrt.

٢. فتح القوس بعد اسم الدالة وإغلاقه بعد كتابة الرقم.

٣. (x) مكان إضافة الرقم.



دالة `sqrt()` تحتاج إلى استدعاء مكتبة `math` لتعمل بشكل صحيح، لذلك نحتاج إلى التعرف على المكتبات في لغة بايثون، من خلال شرح المكتبات سوف نتعرف على استخدام دالة `sqrt()`.

المكتبات في لغة بايثون Modules:

المكتبات في البرمجة عبارة مجموعة من المتغيرات، والدوال، يمكن تضمينها ضمن شيفرة البرمجية الخاصة، حيث قام مجموعة من المطورين بكتابتها ونشرها لتصبح متاحة للجميع والهدف منها تسهيل تنفيذ أمور معينة.

طرق استدعاء المكتبات في لغة بايثون:

١. استدعاء المكتبة بدون دوال (`import x`):

في هذه الطريقة يتم استدعاء المكتبة بدون أي دالة، حيث يتم استدعاء الدوال من خلال تعريف اسم المكتبة قبل اسم الدالة، في هذه الطريقة سوف تعمل جميع الدوال دون الحاجة إلى استدعائها مره أخرى لكن تحتاج لإضافة اسم المكتبة لكل دالة.

```
import x
  ↑   ↑
  1   2
```

١. في بداية الشيفرة البرمجية كتابة كلمة `import` في البداية.

٢. إضافة مسافة ثم كتابة اسم المكتبة مكان (`x`).

مثال:

قم باستدعاء مكتبة `math` ثم احسب الجذر التربيعي للرقم (4) بواسطة دالة `sqrt()`.

```
import math
print(math.sqrt(4))
```

النتيجة:

```
2.0
>>>
```

كما تلاحظ في المثال تم استدعاء مكتبة `math` في البداية ثم استدعاء دالة `sqrt()` لكن تم إضافة كلمة `math` قبل اسم الدالة، وذلك للإشارة إلى المكتبة المطلوب استيراد الدالة منها، ثم الطباعة باستخدام دالة `print()`.



تنويه:

يمكن اختصار اسم المكتبة بدلا عن الاستدعاء بهذه الخطوة من خلال كتابة

الأمر:

```
import math as m
↑      ↑      ↑      ↑
1      2      3      4
```

١. في بداية الشيفرة البرمجية كتابة كلمة import في البداية.

٢. إضافة مسافة ثم كتابة اسم المكتبة مكان (x).

٣. كتابة كلمة as كإشارة للاسم الجديد.

٤. كتابة اسم المكتبة المختصر مكان حرف (m).

٢. استدعاء المكتبة مع الدوال (from x import *):

في هذه الطريقة يتم استدعاء المكتبة مع جميع الدوال، حيث يتم استدعاء الدوال مباشرة دون الحاجة للإشارة إلى اسم المكتبة، في هذه الطريقة سوف تعمل جميع الدوال دون الحاجة إلى استدعائها مره أخرى.

```
from x import *
↑      ↑      ↑      ↑
1      2      3      4
```

١. في بداية الشيفرة البرمجية كتابة كلمة from في البداية.

٢. إضافة مسافة ثم كتابة اسم المكتبة مكان (x).

٣. إضافة مسافة ثم كتابة كلمة import.

٤. إضافة مسافة ثم إضافة علامة الضرب (*).

مثال:

قم باستدعاء المكتبة math مع جميع الدوال ثم احسب الجذر التربيعي للرقم (4)

بواسطة دالة sqrt().

```
from math import *
print(sqrt(4))
```

النتيجة:

```
2.0
>>>
```



كما تلاحظ في المثال تم استدعاء مكتبة `math` في البداية بالإضافة إلى استيراد جميع الدوال وذلك بوضع علامة (*) ثم استدعاء دالة `sqrt()` مباشرة بدون إشارة للمكتبة، ثم الطباعة باستخدام دالة `print()`.

٣. استدعاء دالة معينة من المكتبة `(from x import name)`:

في هذه الطريقة يتم استدعاء دالة معينة من المكتبة، حيث يتم استدعاء الدالة مباشرة دون الحاجة للإشارة إلى اسم المكتبة، في هذه الطريقة لن تعمل أي دالة أخرى إلا بعد أن يتم استدعائها من جديد.

```
from x import name
↑   ↑   ↑   ↑
1   2   3   4
```

١. في بداية الشيفرة البرمجية كتابة كلمة `from` في البداية.

٢. إضافة مسافة ثم كتابة اسم المكتبة مكان (x).

٣. إضافة مسافة ثم كتابة كلمة `import`.

٤. إضافة مسافة ثم كتابة اسم الدالة مكان (name).

مثال:

قم باستدعاء دالة `sqrt()` فقط من مكتبة `math`، ثم احسب الجذر التربيعي للرقم (4).

```
from math import sqrt
print(sqrt(4))
```

النتيجة:

```
2.0
>>>
```

كما تلاحظ في المثال تم استدعاء مكتبة `math` في البداية بالإضافة إلى استيراد دالة `sqrt` فقط، ثم استدعاء دالة `sqrt()` مباشرة بدون إشارة للمكتبة، ثم الطباعة باستخدام دالة `print()`.



معاملات الإسناد الرياضية:

في لغة بايثون يوجد خمس معاملات إسناد رياضية هي:

المعامل	الوظيفة
+=	إضافة قيمة محددة لقيمة المتغير السابقة.
-+	إنقاص قيمة محددة من قيمة المتغير السابقة.
*=	ضرب قيمة محددة في قيمة المتغير السابقة.
/=	قسمة قيمة محددة على قيمة المتغير السابقة.
%=	حساب باقي القسمة لقيمة محددة من قيمة المتغير السابقة.

في هذه الحقبة سوف نتعرف على معاملين مهمين (+=) و (-=)، وذلك لاستخدامهم مستقبلاً في درس التكرار loop.

١. معاملة +=:

يقوم المعامل += بإضافة قيمة محددة إلى قيمة متغير سابقة، حيث لا يقوم بحذف أو استبدال القيمة المسندة إلى المتغير مسبقاً.

مثال:

اكتب برنامج يطلب من المستخدم رقم ثم إضافته لقيمة المتغير X حيث $x=3$ ، ثم يقوم بطباعة قيمة المتغير قبل وبعد الإضافة.

```
a=float(input('Enter Num: '))
x=3
print(x)
x += a
print(x)
```

النتيجة:

```
Enter Num: 4.5
x: 3
New x: 7.5
```

كما تلاحظ في المثال تم استخدام دالة input() لطلب رقم من المستخدم، ثم تعريف المتغير X بالقيمة المحددة (3)، ثم طباعة قيمة المتغير x، ثم تنفيذ معاملة الإضافة +=، ثم طباعة قيمة المتغير x بعد الإضافة.



٢. معامِل (-=):

يقوم المعامِل (-=) بإنقاص قيمة محددة إلى قيمة متغير سابقة، حيث لا يقوم بحذف أو استبدال القيمة المسندة إلى المتغير مسبقاً.

مثال:

اكتب برنامج يطلب من المستخدم رقم ثم ينقصه من قيمة المتغير x حيث $x=5$ ، ثم يقوم بطباعة قيمة المتغير قبل وبعد الإضافة.

```
a=float(input('Enter Num: '))
x=5
print(x)
x -= a
print(x)
```

النتيجة:

```
Enter Num: 2
x: 5
New x: 3.0
```

كما تلاحظ في المثال تم استخدام دالة `input()` لطلب رقم من المستخدم، ثم تعريف المتغير x بالقيمة المحددة (5)، ثم طباعة قيمة المتغير x ، ثم تنفيذ معامِل الإنقاص (-=)، ثم طباعة قيمة المتغير x بعد الإنقاص.



تمارين الوحدة

١. اكتب برنامج يطلب من المستخدم ثلاثة أرقام ويطلع حاصل جمعهم.
٢. اكتب برنامج يقوم بحساب حاصل ضرب عددين وطباعة الناتج.
٣. اكتب برنامج يطلب من المستخدم رقمين ويطلع باقي قسمتهما.
٤. اذكر ترتيب أولويات العمليات الحسابية في لغة بايثون.
٥. اكتب برنامج يطلب من المستخدم ثلاثة أرقام ويطلع أصغر رقم.
٦. اكتب برنامج يطلب من المستخدم ثلاث كلمات ويطلع الكلمة التي تبدأ بأخر حرف.
٧. اكتب برنامج يوضح عمل الدوال التالية:
 - pow()
 - round()
٨. اذكر اثنين من معاملات الإسناد الرياضية مع كتابة برنامج يوضح عمل كل معامل.
٩. اكتب التعبيرات الرياضية التالية بلغة بايثون:

$$2 - \frac{2x - 1}{3} + \frac{1 - 3x}{7}$$

$$\frac{8x + 3}{5} - \frac{11x - 9}{6} + \frac{4x + 3}{15}$$

١٠. اوجد ناتج المعادلات حيث $x=2$:

$$\frac{3x - 4}{3} - \frac{5x - 1}{9} =$$

$$\frac{3x + 8}{2} - 4k =$$



نموذج تقييم المتدرب لمستوى أدائه				
يعبأ من قبل المتدرب نفسه وذلك بعد الانتهاء من تمارين الوحدة				
بعد الانتهاء من التدريب على وحدة العمليات الحسابية، قيم نفسك وقدراتك بواسطة إكمال هذا التقييم الذاتي بعد كل عنصر من العناصر المذكورة، وذلك بوضع علامة (✓) أمام مستوى الأداء الذي أتقنته، وفي حالة عدم قابلية المهمة للتطبيق ضع العلامة في الخانة الخاصة بذلك.				
م	العناصر	مستوى الأداء (هل أتقنت الأداء)		
		غير قابل للتطبيق	لا	جزئياً
١	ينفذ التعبيرات الرياضية في لغة بايثون.			
٢	يستعمل الدوال الرياضية في لغة بايثون.			
٣	يستدعي المكتبة مع دوالها في لغة بايثون.			
٤	يطبق معاملات الإسناد الرياضية في لغة بايثون.			
يجب أن تصل النتيجة لجميع المفردات (البند) المذكورة إلى درجة الإتقان الكلي أو غير قابلة للتطبيق، وفي حالة وجود مفردة في القائمة "لا" أو "جزئياً" فيجب إعادة التدريب على هذا النشاط مرة أخرى بمساعدة المدرب.				



نموذج تقييم المدرب لمستوى أداء المتدرب					
يعبأ من قبل المدرب وذلك بعد الانتهاء من تمارين الوحدة					
اسم المتدرب :		التاريخ:			
رقم المتدرب :		المحاولة: ١ ٢ ٣ ٤			
		العلامة:			
كل بند أو مفردة يقيم ب ١٠ نقاط الحد الأدنى: ما يعادل ٨٠٪ من مجموع النقاط. الحد الأعلى: ما يعادل ١٠٠٪ من مجموع النقاط.					
م	بنود التقييم	النقاط (حسب رقم المحاولات)			
		١	٢	٣	٤
١	ينفذ التعبيرات الرياضية في لغة بايثون.				
٢	يستعمل الدوال الرياضية في لغة بايثون.				
٣	يستدعي المكتبة مع دوالها في لغة بايثون.				
٤	يطبق معاملات الإسناد الرياضية في لغة بايثون.				
المجموع					
ملحوظات:					
.....					
توقيع المدرب:					



الوحدة الخامسة

العمليات العلائقية والنطقية



الوحدة الخامسة

العمليات العلائقية والمنطقية

الهدف العام للوحدة:

تهدف هذه الوحدة إلى استخدام المتدرب العمليات العلائقية والمنطقية واستخدام دوال توليد الأرقام العشوائية في لغة بايثون.

الأهداف التفصيلية:

من المتوقع في نهاية هذه الوحدة التدريبية أن يكون المتدرب قادراً وبكفاءة على أن:

١. يكتب التعبيرات الشرطية والمنطقية في لغة بايثون.
٢. يستخدم دوال الأرقام العشوائية في لغة بايثون.
٣. يشرح أخطاء البرمجة في لغة بايثون.

الوقت المتوقع للتدريب على هذه الوحدة: ١٥ ساعة تدريبية.

الوسائل المساعدة:

١. جهاز حاسب آلي.
٢. جهاز عرض (Data Show).
٣. محرر كتابة Python



معاملات المقارنة:

عبارة عن معاملات تستخدم لمقارنة القيم مع بعضها ، حيث تقوم بمقارنة القيم من حيث الأكبر أو الأصغر أو التساوي ، في لغة بايثون هنالك أربع معاملات للمقارنة: (<) و(<=) و(>) و(>=).

١. معامل أصغر من (<):

هو رمز رياضي يدل على عدم المساواة بين قيمتين. ويستخدم للدلالة على أن الطرف الأيسر في المتباينة أصغر من الطرف الأيمن.

A	<	B
↑	↑	↑
1	2	3

١. كتابة قيمة الطرف الأيسر مكان (A).

٢. إضافة مسافة قبل وبعد علامة المقارنة (<).

٣. كتابة قيمة الطرف الأيمن مكان (B).

مثال:

اكتب برنامج يطلب من المستخدم إدخال رقم صحيح ، ثم يقارنه بالرقم (6) ، إذا كان الرقم المدخل أصغر منه يطبع (True).

```
x=int(input('Enter Num: '))
r=x < 6
print(r)
```

النتيجة:

```
Enter Num: 5
True
```

عند إدخال الرقم (5) تحقق شرط المقارنة ، بمعنى أن رقم (5) أصغر من الرقم (6) ، لذلك تمت طباعة (True).

```
Enter Num: 8
False
```

عند إدخال الرقم (8) لم يتحقق شرط المقارنة ، بمعنى أن رقم (8) أكبر من الرقم (6) ، لذلك تمت طباعة (False).



٢. معامل أصغر من أو يساوي ($\leq$):

هو رمز رياضي يدل على عدم المساواة أو المساواة بين قيمتين. ويستخدم للدلالة على أن الطرف الأيسر في المتباينة أصغر من الطرف الأيمن أو يساويه.

A	$\leq$	B
↑	↑	↑
1	2	3

١. كتابة قيمة الطرف الأيسر مكان (A).

٢. إضافة مسافة قبل وبعد علامة المقارنة ($\leq$).

٣. كتابة قيمة الطرف الأيمن مكان (B).

مثال:

اكتب برنامج يطلب من المستخدم إدخال رقم صحيح، ثم يقارنه بالرقم (6)، إذا كان الرقم المدخل أصغر منه أو يساويه يطبع (True).

```
x=int(input('Enter Num: '))
r=x <= 6
print(r)
```

النتيجة:

```
Enter Num: 6
True
```

عند إدخال الرقم (6) تحقق شرط المقارنة، بمعنى أن رقم (6) مساوي للرقم (6)، لذلك تمت طباعة (True).

```
Enter Num: 3
True
```

عند إدخال الرقم (3) تحقق شرط المقارنة، بمعنى أن رقم (3) أصغر من الرقم (6)، لذلك تمت طباعة (True).

```
Enter Num: 7
False
```

عند إدخال الرقم (7) لم يتحقق شرط المقارنة، بمعنى أن رقم (7) أكبر من الرقم (6)، لذلك تمت طباعة (False).



٣. معامل أكبر من (>):

هو رمز رياضي يدل على عدم المساواة بين قيمتين. ويستخدم للدلالة على أن الطرف الأيسر في المتباينة أكبر من الطرف الأيمن.

A	>	B
↑	↑	↑
1	2	3

١. كتابة قيمة الطرف الأيسر مكان (A).

٢. إضافة مسافة قبل وبعد علامة المقارنة (>).

٣. كتابة قيمة الطرف الأيمن مكان (B).

مثال:

اكتب برنامج يطلب من المستخدم إدخال رقم صحيح، ثم يقارنه بالرقم (6)، إذا كان الرقم المدخل أكبر منه يطبع (True).

```
x=int(input('Enter Num: '))
r=x > 6
print(r)
```

النتيجة:

```
Enter Num: 9
True
```

عند إدخال الرقم (9) تحقق شرط المقارنة، بمعنى أن رقم (٩) أكبر من الرقم (6)، لذلك تمت طباعة (True).

```
Enter Num: 5
False
```

عند إدخال الرقم (5) لم يتحقق شرط المقارنة، بمعنى أن رقم (5) أصغر من الرقم (6)، لذلك تمت طباعة (False).

٤. معامل أكبر من أو يساوي (>=):

هو رمز رياضي يدل على عدم المساواة أو المساواة بين قيمتين. ويستخدم للدلالة على أن الطرف الأيسر في المتباينة أكبر من الطرف الأيمن أو يساويه.

A	>=	B
↑	↑	↑
1	2	3



١. كتابة قيمة الطرف الايسر مكان (A).

٢. إضافة مسافة قبل وبعد علامة المقارنة ($\geq$).

٣. كتابة قيمة الطرف الأيمن مكان (B).

مثال:

اكتب برنامج يطلب من المستخدم إدخال رقم صحيح، ثم يقارنه بالرقم (6)، إذا كان الرقم المدخل أصغر منه يطبع (True).

```
x=int(input('Enter Num: '))
r=x >= 6
print(r)
```

النتيجة:

```
Enter Num: 8
True
```

عند إدخال الرقم (8) تحقق شرط المقارنة، بمعنى أن رقم (8) أكبر من الرقم (6)، لذلك تمت طباعة (True).

```
Enter Num: 6
True
```

عند إدخال الرقم (6) تحقق شرط المقارنة، بمعنى أن رقم (6) مساوي للرقم (6)، لذلك تمت طباعة (True).

```
Enter Num: 3
False
```

عند إدخال الرقم (3) لم يتحقق شرط المقارنة، بمعنى أن رقم (3) أصغر من الرقم (6)، لذلك تمت طباعة (False).

معاملات المساواة:

عبارة عن معاملات مقارنة حيث تقوم بمقارنة القيم من حيث التساوي أو عدم التساوي، في لغة بايثون يوجد معاملين للمساواة: ($==$) و ($!=$).

١. معامل المساواة ($==$):

عبارة عن معامل يقوم بمقارنة القيمة هل تساوي القيمة الأخرى، حيث يكون الجواب صح أو خطأ فقط.



```

A == B
↑  ↑  ↑
1  2  3

```

١. القيمة الأول تكون مكان (A).

٢. إضافة مسافة ثم علامة (==) ثم مسافة.

٣. القيمة الثانية تكون مكان (B).

مثال:

اكتب برنامج يطلب إدخال قيمة من المستخدم، ثم يقارنها بالتساوي مع الرقم (5)، وطباعة النتيجة.

```

x=int(input('1st Num: '))
r= x == 5
print(r)

```

النتيجة:

```

1st Num: 5
True

```

في حال إدخال قيمة تساوي الرقم (5) فإن النتيجة تكون (True).

```

1st Num: 3
False

```

في حال إدخال قيمة لا تساوي الرقم (5) فإن النتيجة تكون (False).

معامل عدم المساواة (!=):

عبارة عن معامل يقوم بمقارنة القيمة هل هي مساوية للقيمة الأخرى، حيث

يكون الجواب صح أو خطأ فقط.

```

A != B
↑  ↑  ↑
1  2  3

```

١. القيمة الأول تكون مكان (A).

٢. إضافة مسافة ثم علامة (!=) ثم مسافة.

٣. القيمة الثانية تكون مكان (B).



مثال:

اكتب برنامج يطلب إدخال قيمة من المستخدم، ثم يقارنها بعدم التساوي مع الرقم (5)، وطباعة النتيجة.

```
x=int(input('Enter Num: '))
r= x != 5
print(r)
```

النتيجة:

```
Enter Num: 5
False
```

في حال إدخال قيمة تساوي الرقم (5) فإن النتيجة تكون (False).

```
Enter Num: 8
True
```

في حال إدخال قيمة لا تساوي الرقم (5) فإن النتيجة تكون (True).

المعاملات المنطقية في لغة بايثون:

القيم المنطقية ترمز إلى الأشياء التي لا تحدث أكثر من احتمالين وهما إما صح وإما خطأ، حيث تساعدنا في صناعة الشروط والقيود على شيء معين وبالتالي تمنحنا تحكماً أكبر في الشيفرة البرمجية.

المعاملات المنطقية في لغة بايثون تنقسم إلى ثلاثة أقسام هي: and و or و not، فكل معامل منها له نتائج مختلفة، في هذه الحقيبة سوف نركز على معاملي and، or.

١. المعامل and:

تعني الجمع بين شرطين، عند استخدام المعامل and لابد من تحقق الشرطين للحصول على النتيجة الصحيحة، مثال: إذا الرقم 5 أكبر من الرقم 2 (and) الرقم 3 أصغر من الرقم 4، تكون النتيجة صحيحة لتحقيق الشرطين.

A	and	B
↑	↑	↑
1	2	3

١. تتم كتابة الشرط الأول مكان (A).

٢. تتم إضافة مسافة قبل وبعد معاملي (and).

٣. تتم كتابة الشرط الثاني مكان (B).



مثال:

اكتب برنامج يطلب من المستخدم إدخال رقمين صحيحين في المتغيرين x و y ثم يقارن قيمتهما مع الرقم 5، حيث x أكبر من 5 و y أصغر من 5، ثم يطبع النتيجة.

```
x=int(input('1st Num: '))
y=int(input('2nd Num: '))
r= x>5 and y<5
print(r)
```

النتيجة:

```
1st Num: 6
2nd Num: 3
True
```

كما تلاحظ عند إدخال أرقام تحقق الشرط تتم طباعة (True)، بمعنى الأرقام المدخلة حققت الشروط المحددة.

```
1st Num: 3
2nd Num: 6
False
```

كما تلاحظ عند إدخال أرقام لا تحقق الشرط تتم طباعة (False)، بمعنى الأرقام المدخلة لم تحقق الشروط المحددة.

٢. العامل or:

يعني اختيار أحد الشرطين، عند استخدام العامل or لابد من تحقق شرط واحد على الأقل للحصول على النتيجة الصحيحة، مثال: إذا الرقم 5 أكبر من الرقم 2 (or) الرقم 3 أصغر من الرقم 4، تكون النتيجة صحيحة لتحقيق الشرطين.

A	or	B
↑	↑	↑
1	2	3

١. تتم كتابة الشرط الأول مكان (A).

٢. تتم إضافة مسافة قبل وبعد عامل (or).

٣. تتم كتابة الشرط الثاني مكان (B).

مثال:

اكتب برنامج يطلب من المستخدم إدخال رقمين صحيحين في المتغيرين x و y ثم يقارن قيمتهما مع الرقم 5، حيث x أكبر من 5 و y أصغر من 5، ثم يطبع النتيجة.



```
x=int(input('1st Num: '))
y=int(input('2nd Num: '))
r= x>5 or y<5
print(r)
```

النتيجة:

```
1st Num: 6
2nd Num: 3
True
```

كما تلاحظ عند إدخال أرقام تحقق الشرط تتم طباعة (True)، بمعنى أن الأرقام المدخلة حققت الشروط المحددة.

```
1st Num: 3
2nd Num: 4
True
```

كما تلاحظ عند إدخال أرقام تحقق شرط واحد فقط تم طباعة (True)، بمعنى أن أحد الأرقام المدخلة حقق أحد الشروط المحددة.

```
1st Num: 3
2nd Num: 6
False
```

كما تلاحظ عند إدخال أرقام لا تحقق الشرطين تتم طباعة (False)، بمعنى أن الأرقام المدخلة لم تحقق الشروط المحددة.

مقارنة بين and و or:

المقارنة	or	and
الشروط	2 فأكثر	2 فأكثر
طباعة (True)	تحقق شرط واحد	تحقق الشرطين
طباعة (False)	عدم تحقق الشرطين	عدم تحقق شرط واحد
الوظيفة	اختيار	جمع

كتابة التعبيرات الشرطية والمنطقية

في لغة بايثون يمكن كتابة التعبيرات الشرطية أو المنطقية مستقلة أو الجمع بينهم، حيث يمكن كتابة أكثر من تعبير شرطي وربطهم بتعبير منطقي.



استخدام تعبير شرطي واحد.

```
x > z
z < x
o == x
z != o
```

استخدام تعبير منطقي واحد.

```
a or b
a and b
```

استخدام تعبيرين شرطين.

```
x >= z
z <= x
```

استخدام أكثر من تعبير شرطي وتعبير منطقي.

```
x > o and z < o
x == o or z != o
```

استخدام أكثر من تعبيرين شرطين وأكثر من تعبير منطقي.

```
x == z and s != z and w > z
x == z and s != z or w > z
x == z or s != z and w > z
x == z or s != z or w > z
```

توليد الأرقام العشوائية

الأرقام العشوائية عبارة عن متتالية من الأعداد تفتقر إلى أي نظام أو ترتيب. وبتعبير آخر هي أعداد عشوائية.

في لغة البرمجة بايثون يتم توليد الأرقام العشوائية من خلال بعض الدوال، في هذه الحقبة سوف نتعلم دالتين هما: `randint()` و `random()`.

تنبيه:

دوال توليد الأرقام العشوائية تحتاج استدعاء مكتبة `random`.

١. دالة `randint()`:

هي عبارة عن دالة جاهزة في لغة بايثون تقوم بتوليد أرقام عشوائية صحيحة وذلك ضمن نطاق يحدد من قبل المبرمج أو المستخدم.



```
from random import randint ← 1
randint(start, stop)
    ↑   ↑   ↑   ↑   ↑
    2   3   4   5   6
```

١. استدعاء دالة randint من مكتبة random.

٢. كتابة اسم الدالة في البداية.

٣. فتح القوس ثم اغلاقه بعد الانتهاء.

٤. كتابة رقم بداية التوليد.

٥. وضع علامة فاصلة.

٦. كتابة رقم نهائي التوليد.

مثال:

اكتب برنامج يقوم بتوليد رقم عشوائي بين 5 و10، ويطبع النتيجة على الشاشة.

```
from random import randint
x=randint(5, 10)
print(x)
print(randint(5, 10))
```

النتيجة:

```
9
8
```

كما تلاحظ في المثال تمت طباعة نتيجة دالة randint مرتين،

الأولى من خلال تعرف متغير x ثم طباعة قيمة المتغير،

الثانية من خلال تنفيذ دالة randint في دالة print،

النتائج في كليهما مختلفة وهذا يوضح لك أنه في كل مرة يتم استدعاء دالة

randint يتم إظهار رقم مختلف عن سابقه .

٢. دالة random():

هي عبارة عن دالة جاهزة في لغة بايثون تقوم بتوليد أرقام عشوائية عشرية عشوائية

دون تحديد نطاق التوليد.



```
from random import random ← 1
random()
↑   ↑
2   3
```

١ . استدعاء دالة random من مكتبة random.

٢ . كتابة اسم الدالة في البداية.

٣ . فتح علامة القوس واغلاقه مباشرة.

مثال:

اكتب برنامج يقوم بتوليد رقم عشوائي عشري، ويطبع النتيجة على الشاشة.

```
from random import random
x=random()
print(x)
print(random())
```

النتيجة:

```
0.7982738840029308
0.716135183101333
```

كما تلاحظ في المثال تم طباعة نتيجة دالة random مرتين،

الأولى من خلال تعرف متغير x ثم طباعة قيمة المتغير،

الثانية من خلال تنفيذ دالة random في دالة print،

النتائج في كلتيهما مختلفة وهذا يوضح لك أنه في كل مرة يتم استدعاء دالة

random يتم إظهار رقم مختلف عن سابقه.

الأخطاء في البرمجة:

الخطأ مصطلح يستخدم لوصف مشكلة برمجية تظهر بشكل غير متوقع وتسبب في

توقف عمل الشيفرة البرمجية، في لغات البرمجة تصنف الأخطاء إلى ثلاثة أنواع رئيسية هي:

١ . أخطاء لغوية (Syntax Errors):

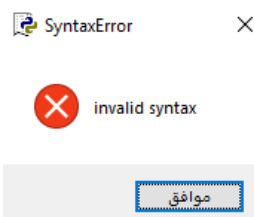
هي الأخطاء التي تقع أثناء كتابة الشيفرة البرمجية من قبل المبرمج، وهي عبارة عن

أخطاء في قواعد كتابة لغة البرمجة مثلاً: عدم إغلاق القوس بعد الانتهاء أو عدم استخدام

الفاصلة الصحيحة أو وضع مسافات في الكلمات المحجوزة.

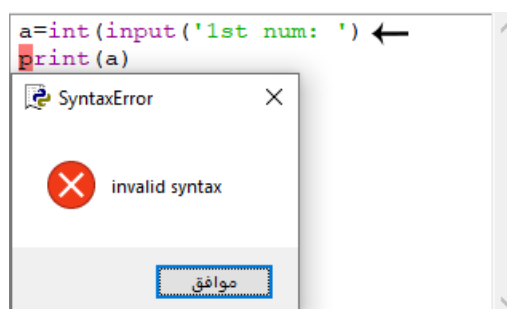


صورة لرسالة الخطأ:



الشكل (٥ - ١)

مثال:



الشكل (٥ - ٢)

رسالة الخطأ أعلاه ظهرت لعدم إغلاق القوس، كما تلاحظ تم فتح القوس لدالة int() ثم فتح القوس لدالة input() ولم يتم إغلاق إلا قوساً واحداً فقط.

٢. أخطاء منطقية (Logic Errors):

هي الأخطاء التي تقع لعدم فهم طريقة عمل الشيفرة البرمجية، حيث لا يظهر هذا الخطأ إلا بعد عمل الشيفرة البرمجية، ويعتبر من أصعب الأخطاء اكتشافاً؛ لأنه يستغرق وقتاً لحدوثه، وفي بعض الأحيان يكتشف من قبل مستخدمي البرنامج، مثل: وضع علامة الطرح بدلاً من علامة الجمع أو تنفيذ عملية قبل الأخرى.

مثال:

اكتب برنامج يطلب رقمين ويطبّع حاصل جمعهما.

```
a=int(input('1st Num: '))
b=int(input('2nd Num: '))
print(a-b)
```

النتيجة:

```
1st Num: 5
2nd Num: 3
2
```



كما تلاحظ الشيفرة البرمجية تعمل بشكل صحيح، حيث طلب من المستخدم إدخال رقمين، لكن تم تنفيذ عملية الطرح بدلاً من الجمع، وهنا يكون الخطأ المنطقي، ثم طباعة النتيجة، وفي هذه الحالة لن تكتشف الخطأ إلا بعد تجربة البرنامج والتدقيق في المدخلات والمخرجات، هل هي مطابقة لما هو مطلوب أم لا؟
لذلك ينصح بتجربة البرنامج قبل عرضه على المستخدم، أو إطلاق نسخة تجريبية وطلب تغذية راجعة من المستخدمين بعد التجربة لتلافي مثل هذه الأخطاء.

٣. أخطاء وقت التشغيل (Run-Time Errors):

هي الأخطاء التي تقع أثناء تشغيل الشيفرة البرمجية تحديداً عند إدخال البيانات للبرنامج، وتعتبر من أكثر الأخطاء حدوثاً مثل: تحديد رقم صحيح لمتغير رقمي، فعند إدخال رقم عشري سوف يظهر خطأ برمجي أو إدخال نصوص في متغير رقمي، أو طباعة متغير لم يتم تعريفه مسبقاً.

مثال:

اكتب برنامج يطلب من المستخدم رقم ثم يطبعه على الشاشة.

```
a=int(input('1st Num: '))
print(a)
```

النتيجة:

```
1st Num: 3.5
Traceback (most recent call last):
  File "C:/Users/Tam/AppData/Local/Programs/Python/Python38-32/123.py", line 1, in <module>
    a=int(input('1st Num: '))
ValueError: invalid literal for int() with base 10: '3.5'
```

كما تلاحظ عند إدخال رقم عشري ظهر خطأ برمجي، وذلك بسبب تحديد نوع البيانات المدخلة بأرقام صحيحة فقط، وتعتبر هذه الأخطاء سهلة مقارنة بالأخطاء السابقة وذلك لقيام محرر كتابة لغة بايثون بتحديد موقع الخطأ ونوعه، لذلك يسهل اكتشاف ومعالجة الخطأ قبل وقت تنفيذ البرنامج لفحصه.



تمارين الوحدة

١. اكتب برنامج يطلب من المستخدم إدخال رقمين ليقارنهم وفي حالة التساوي يطبع (True).
٢. اكتب برنامج يطلب من المستخدم رقمين ويسندهم للمتغيرات x و o ثم يطبق الشرط $x > 0$ ، ثم يطبع النتيجة.
٣. اكتب برنامج يطلب من المستخدم إدخال رقم عشري ثم يقوم البرنامج بمقارنته مع الرقم (8.5)، فإذا كان الرقم المدخل كان أكبر أو يساوي (8.5) يطبع (True).
٤. اكتب برنامج يطلب من المستخدم إدخال رقمين ليقوم البرنامج باسنادهم للمتغيرات a ، b ، حيث $a > 5$ و $b < 3$ ، ثم يقوم بطباعة الناتج، (العلاقة بين الشرطين and).
٥. اكتب برنامج يطلب من المستخدم ثلاثة أرقام بالتوالي $a > b$ و $b > c$ و $a > c$ ، ثم يطبع النتيجة.
٦. اكتب برنامج يقوم بتوليد أرقام عشوائية بناء على تحديد المستخدم.
٧. اكتب برنامج يقوم بتوليد أرقام عشوائية عشرية.
٨. اذكر أنواع الأخطاء البرمجية، مع ذكر مثال لكل منها.



نموذج تقييم المتدرب لمستوى أدائه					
يعبأ من قبل المتدرب نفسه وذلك بعد الانتهاء من تمارين الوحدة					
بعد الانتهاء من التدريب على وحدة العمليات العلائقية والمنطقية، قيم نفسك وقدراتك بواسطة إكمال هذا التقييم الذاتي بعد كل عنصر من العناصر المذكورة، وذلك بوضع علامة (✓) أمام مستوى الأداء الذي أتقنته، وفي حالة عدم قابلية المهمة للتطبيق ضع العلامة في الخانة الخاصة بذلك.					
م	العناصر	مستوى الأداء (هل أتقنت الأداء)			
		غير قابل للتطبيق	لا	جزئيا	كليا
١	يكتب التعبيرات الشرطية والمنطقية في لغة بايثون.				
٢	يستخدم دوال الأرقام العشوائية في لغة بايثون.				
٣	يشرح أخطاء البرمجة في لغة بايثون.				
يجب أن تصل النتيجة لجميع المفردات (البندود) المذكورة إلى درجة الإتقان الكلي أو غير قابلة للتطبيق، وفي حالة وجود مفردة في القائمة "لا" أو "جزئيا" فيجب إعادة التدريب على هذا النشاط مرة أخرى بمساعدة المدرب.					



نموذج تقييم المدرب لمستوى أداء المتدرب					
يعبأ من قبل المدرب وذلك بعد الانتهاء من تمارين الوحدة					
اسم المتدرب :		التاريخ:			
رقم المتدرب :		المحاولة: ١ ٢ ٣ ٤			
		العلامة:			
كل بند أو مفردة يقيم ب ١٠ نقاط الحد الأدنى: ما يعادل ٨٠٪ من مجموع النقاط. الحد الأعلى: ما يعادل ١٠٠٪ من مجموع النقاط.					
م	بنود التقييم	النقاط (حسب رقم المحاولات)			
		١	٢	٣	٤
١	يكتب التعبيرات الشرطية والمنطقية في لغة بايثون.				
٢	يستخدم دوال الأرقام العشوائية في لغة بايثون.				
٣	يشرح أخطاء البرمجة في لغة بايثون.				
المجموع					
ملحوظات:					
توقيع المدرب:					

الوحدة السادسة

الاختيار بالجمل الشرطية



الوحدة السادسة

الاختيار بالجمل الشرطية

الهدف العام للوحدة:

تهدف هذه الوحدة إلى استخدام المتدرب الجمل الشرطية ورسم مخططاتها الانسيابية في بايثون.

الأهداف التفصيلية:

من المتوقع في نهاية هذه الوحدة التدريبية أن يكون المتدرب قادراً وبكفاءة على أن:

١. يستخدم الجمل الشرطية if-else ، if-elif-else ، في لغة بايثون.
٢. يرسم المخططات الانسيابية للجمل الشرطية.
٣. ينفذ الجمل الشرطية باستخدام دوال المقارنة.

الوقت المتوقع للتدريب على هذه الوحدة: ١٥ ساعة تدريبية.

الوسائل المساعدة:

١. جهاز حاسب آلي.
٢. جهاز عرض (Data Show).
٣. محرر كتابة Python.
٤. برنامج رسم مخططات انسيابية مثل Microsoft Visio.



الجمل الشرطية في لغة بايثون:

عبارة عن أدوات تستخدم لتحديد طريقة عمل البرنامج، حيث يمكن وضع أكثر من شرط في نفس البرنامج أو أكثر من شرط داخل الشرط الأساسي، وتنقسم الجمل الشرطية في لغة بايثون إلى ثلاثة أقسام رئيسة هي:

١. الجملة الشرطية if.
٢. الجملة الشرطية if-else.
٣. الجملة الشرطية if-elif-else.

المخطط الانسيابي:

هو عبارة عن شكل توضيحي باستخدام رموز معينة، يتم رسمه من قبل المبرمج ليوضح طريقة عمل البرنامج كاملاً أو جزء منه.

عوامل المقارنة التي تستخدم في الجمل الشرطية:

هي عبارة عن عوامل رياضية تستخدم لصياغة الشروط حيث تقوم بمقارنة المعطيات للتأكد من تحقيق الشروط حسب المطلوب.

الجدول التالي يوضح عوامل المقارنة ومعانيها:

الرمز	الوظيفة
==	مقارنة قيمة المتغيرات بالتساوي
!=	مقارنة قيمة المتغيرات بغير التساوي
>	مقارنة قيمة المتغير أكبر من
<	مقارنة قيمة المتغير أصغر من
>=	مقارنة قيمة المتغير أكبر من أو يساوي
<=	مقارنة قيمة المتغير أصغر من أو يساوي



الجملة الشرطية if:

بمعنى (إذا) وهي جملة تستخدم عندما يكون الشرط مرتبط بحدث واحد فقط، حيث تسمح للمبرمج بوضع شرط معين إذا تحقق يتم تنفيذ الحدث أو البرنامج، أما إذا كان عكس ذلك فلا يتم تنفيذ شيء أو الخروج من البرنامج، مثل وضع شرط العمر للسماح باستخدام البرنامج.

صيغة كتابة جملة if:

يتم كتابة الجملة الشرطية if في بايثون بالشكل التالي:

```
if condition :
    statements
|
```

الشرح:

```

1 2 3
File Edit Format Run Options Window Help
if condition :
    statement ← 4
| ← 5

```

١. يتم كتابة if في بداية السطر.
٢. تتم إضافة مسافة بعد if ثم كتابة الشرط.
٣. بعد الانتهاء من الشرط يجب وضع نقطتين رأسيتين ثم الضغط على مفتاح Enter.
٤. هنا تتم كتابة الأحداث الخاصة في الشرط if.
٥. يتم إعادة المؤشر لبداية السطر للخروج من الشرط if.

مثال على جملة if:

اكتب برنامج يقوم بطلب عمر المستخدم ثم التأكد من أهليته للحصول على رخصة قيادة، شرط الأهلية أن يكون العمر 18 سنة وأكثر.

```
x=int(input("Enter Age: "))
if x >= 18 :
    print("مؤهل للحصول على رخصة")
```



النتيجة عند إدخال الرقم 19

```
Enter Age: 19
مؤهل للحصول على رخصة
>>>
```

النتيجة عند إدخال الرقم 16:

```
Enter Age: 16
>>>
```

كما تلاحظ عند عدم تحقق الشرط المطلوب، لم يتم تنفيذ أي حدث.

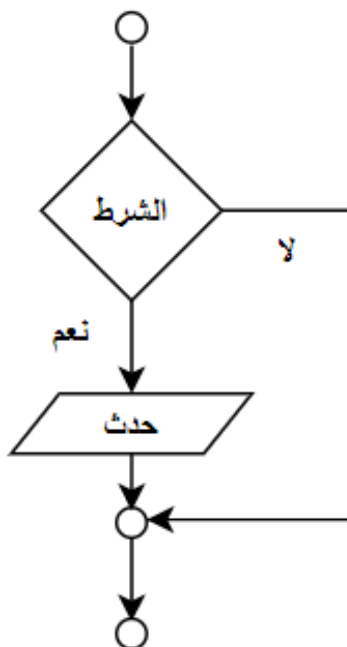
المخطط الانسيابي للشرط if:

الشكل التالي يوضح المخطط الانسيابي للشرط if مع شرح الخطوات:

١. الشرط: يتم إضافة الشرط المطلوب التقيد به، وغالباً يكون باستخدام العمليات العلاقية والمنطقية.

٢. الحدث: إذا تحقق الشرط (نعم) يتم تنفيذ الحدث المحدد.

إذا لم يتحقق الشرط (لا) يتم تجاوز الحدث.

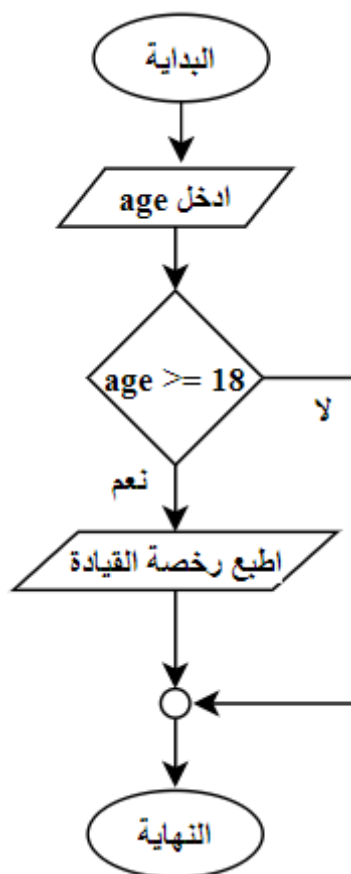


الشكل (٦ - ١)



مثال: ارسم المخطط الانسيابي لبرنامج إصدار رخصة قيادة باستخدام الشرط if، إذا كان العمر 18 سنة أو أكثر يتم إصدار رخصة قيادة.

الحل:



الشكل (٦ - ٢)

الجملة الشرطية if-else:

بمعنى (إما - أو) وهي جملة تستخدم عندما يكون الشرط مرتبط بحدثين مختلفين، حيث لا يتم تنفيذ الحدث الأول إلا بعد تحقق الشرط أو تنفيذ الحدث الثاني في حالة عدم تحقق الشرط، مثل اختيار نتيجة المتدرب ناجح أو راسب.

صيغة كتابة جملة if-else:

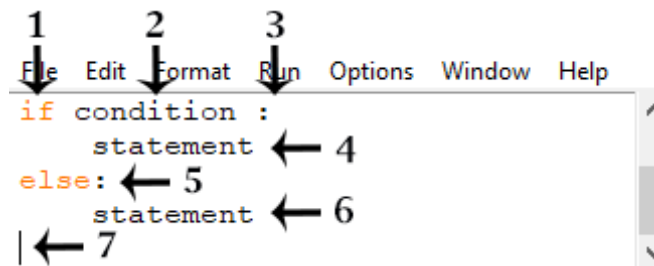
يتم كتابة الجملة الشرطية if-else في بايثون بالشكل التالي:

```

if condition :
    statements
else:
    statements
  
```




الشرح:



١. يتم كتابة if في بداية السطر.
٢. يتم إضافة مسافة بعد if ثم كتابة الشرط.
٣. بعد الانتهاء من الشرط يجب وضع نقطتين رأسيتين ثم الضغط على مفتاح Enter.
٤. هنا يتم كتابة الأحداث الخاصة في الشرط if.
٥. يتم كتابة else في بداية السطر مع نقطتين رأسيتين بعد أحداث if.
٦. هنا يتم كتابة الأحداث الخاصة في else.
٧. يتم إعادة المؤشر لبداية السطر للخروج من الشرط else.

مثال على جملة if-else:

اكتب برنامج يطلب من المستخدم إدخال درجة المتدرب ثم يقوم البرنامج بطباعة عبارة (ناجح) إذا كان ناجحاً أو عبارة (راسب) إذا كان راسباً، مع العلم أن درجة النجاح 50 فأكثر، وغير ذلك راسب.

```
x=int(input("Enter Result: "))
if x>=50 :
    print("ناجح")
else:
    print("راسب")
```

النتيجة عند إدخال 49:

```
Enter Result: 49
راسب
>>>
```

النتيجة عند إدخال 55:

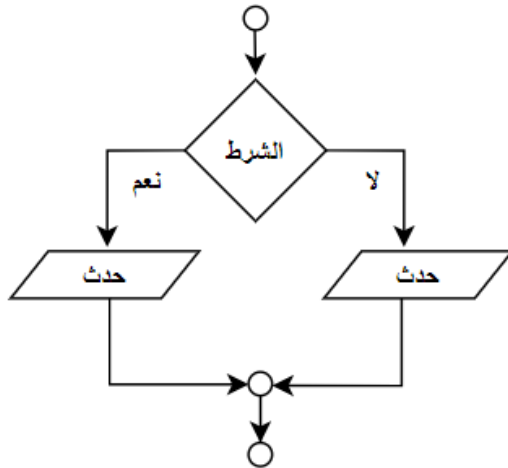
```
Enter Result: 55
ناجح
>>>
```



كما تلاحظ في المرة الأولى عند إدخال رقم لم يتحقق الشرط قام بتنفيذ الحدث المرتبط ب if، وفي المرة الثانية عند إدخال رقم يحقق الشرط قام بتنفيذ الحدث المرتبط ب else.

المخطط الانسيابي للشرط if-else:

الشكل التالي يوضح المخطط الانسيابي للشرط if-else مع شرح الخطوات:



الشكل (٦ - ٣)

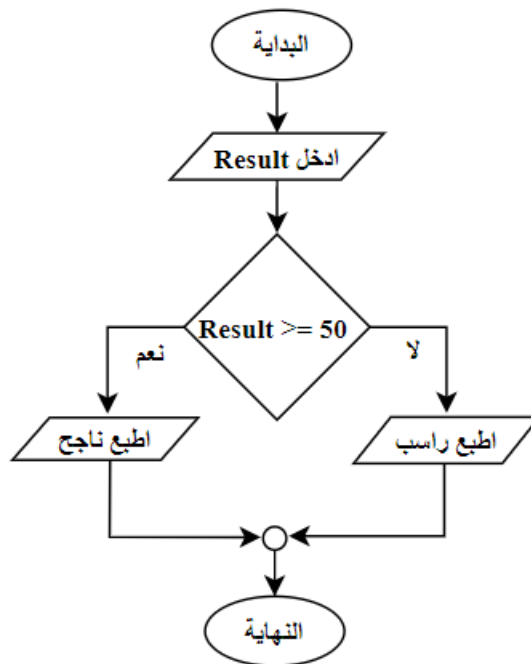
١. الشرط: يتم إضافة الشرط المطلوب التقيد به، وغالباً يكون باستخدام العمليات العلاقية والمنطقية.

٢. الحدث (نعم): إذا تحقق الشرط يتم تنفيذ الحدث المحدد ب نعم.

٣. الحدث (لا): إذا لم يحقق الشرط يتم تنفيذ الحدث المحدد ب لا.

مثال: ارسم المخطط الانسيابي لبرنامج يطبع نتيجة متدرب باستخدام الشرط if-else، بحيث إذا حصل المتدرب على درجة 50 فأكثر يعتبر ناجح أو يطبع راسب.

الحل:



الشكل (٦ - ٤)



الجملة الشرطية if-elif-else:

هي جملة تستخدم عندما يكون هنالك شرطان أو أكثر مرتبطتين بثلاثة أحداث أو أكثر، حيث لا يتم تنفيذ أي حدث إلا بعد تحقق أحد الشروط المرتبطة به أولاً.

صيغة كتابة جملة if- elif -else:

تتم كتابة الجملة الشرطية if-elif-else في بايثون بالشكل التالي:

```
if condition :
    statements
elif condition :
    statements
else:
    statements
|
```

الشرح:

```
File Edit Format Run Options Window Help
1  if condition: 3
   4statements
5  elif condition6:7
   statement 8
9  else:
   statement 10
11 |
```

١. تتم كتابة if في بداية السطر.
٢. تتم إضافة مسافة بعد if ثم كتابة الشرط.
٣. بعد الانتهاء من الشرط يجب وضع نقطتين رأسييتين ثم الضغط على مفتاح Enter.
٤. هنا يتم كتابة الأحداث الخاصة في شرط if.
٥. تتم كتابة elif في بداية السطر بعد أحداث if.
٦. تتم إضافة مسافة بعد elif ثم كتابة الشرط.
٧. بعد الانتهاء من الشرط يجب وضع نقطتين رأسييتين ثم الضغط على مفتاح Enter.
٨. هنا تتم كتابة الأحداث الخاصة في شرط elif.
٩. تتم كتابة else في بداية السطر مع نقطتين رأسييتين بعد أحداث elif.
١٠. هنا تتم كتابة الأحداث الخاصة في else.
١١. تتم إعادة المؤشر لبداية السطر للخروج من الشرط else.



مثال على جملة if-elif-else:

اكتب برنامج يطلب درجة المتدرب ثم يقوم البرنامج بطباعة تقدير المتدرب، بحيث إذا كانت درجة المتدرب 90 فأكثر يحصل على تقدير A، إذا كانت درجته 80 فأكثر يحصل على B، غير ذلك يحصل على تقدير C.

```
x=int(input("Enter Result: "))
if x>=90 :
    print("A")
elif x>=80:
    print("B")
else:
    print("C")
|
```

النتيجة بعد إدخال الرقم 90:

```
Enter Result: 90
A
>>>
```

النتيجة بعد إدخال الرقم 85:

```
Enter Result: 85
B
>>>
```

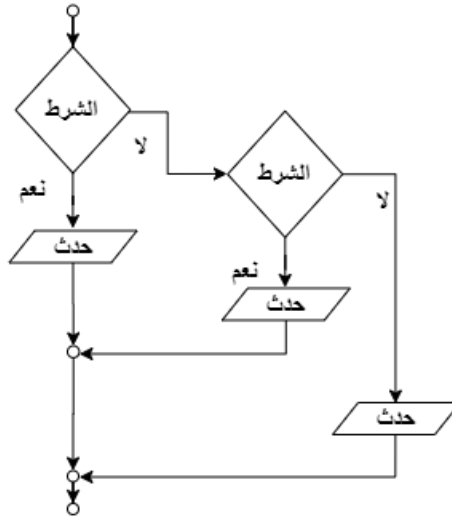
النتيجة بعد إدخال الرقم 50:

```
Enter Result: 50
C
>>>
```

كما تلاحظ عند إدخال أي رقم تتم طباعة النتيجة بناءً على الشرط المتحقق، فعندما تم إدخال الرقم 85 تم تحقق الشرط الثاني وتم تنفيذ الحدث الثاني، وعندما تم إدخال الرقم 55 لم يتحقق أي من الشرطين فتمت طباعة الحدث المرتبط بعدم تحقق الشروط.

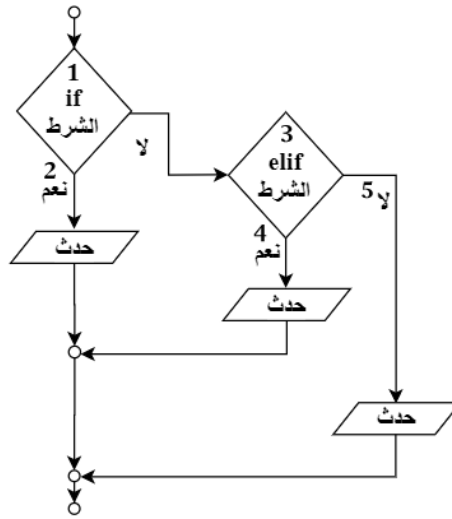


المخطط الانسيابي للشرط if-elif-else:



الشكل (٦ - ٥)

الشرح:



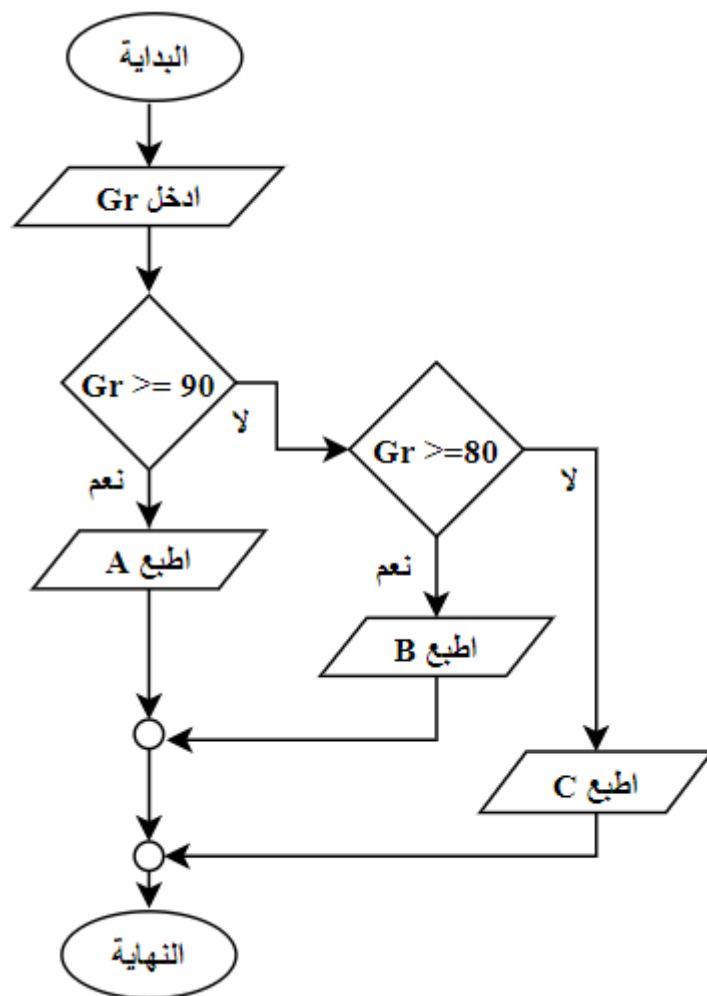
الشكل (٦ - ٦)

١. الشرط (if): تتم إضافة الشرط المطلوب التقيد به، وغالباً يكون باستخدام العمليات العلاقية والمنطقية.
٢. الحدث (نعم): إذا تحقق الشرط يتم تنفيذ الحدث المحدد ب (نعم)، إذا لم يتحقق ينتقل إلى الشرط الثاني.
٣. الشرط (elif): تتم إضافة الشرط المطلوب التقيد به، وغالباً يكون باستخدام العمليات العلاقية والمنطقية.
٤. الحدث (نعم): إذا تحقق الشرط يتم تنفيذ الحدث المحدد ب (نعم).



٥. الحدث: إذا لم يتحقق شرط elif ينتقل إلى الحدث الأخير (else).

مثال: ارسم المخطط الانسيابي لبرنامج يطبع تقدير المتدرب باستخدام الشرط if-elif-else، بحيث إذا حصل المتدرب على درجة 90 يطبع A، و80 يطبع B غير ذلك يطبع C.
الحل:



الشكل (٦ - ٧)

مقارنة بين الجمل الشرطية:

العنصر	if	if-else	if-elif-else
عدد الشروط	1	1	2 فأكثر
عدد الاحداث	1	2	3 فأكثر



الأخطاء الشائعة عند كتابة الشروط في لغة بايثون:

١. عدم إضافة مسافة ذلك يعني خروج الحدث من الشرط if.

```
x = 5
if x == 5:
    print("تحقق الشرط")
```

٢. عدم تساوي المسافة البادئة للأحداث.

```
x = 5
if x == 5:
    print("تحقق الشرط")
    print("Python")
```

يجب وضع مسافة بادئة لأسطر الأحداث لأوامر الشرط (if، if-else، if-elif-else) سواء كان ذلك عن طريق المسافة الواحدة أو زر TAB وذلك لتعمل داخل الشرط.

المعاملات المنطقية التي يمكن استخدامها في الشروط:

هي عبارة عن معاملات منطقية تستخدم لربط عدة شروط بواسطة التعبيرات المنطقية، وهما كالتالي:

- **المعامل and:** وتعني (و) يعني إضافة، تقوم بإضافة أكثر من تعبير لتحقيق الشرط، وعند استخدامها يعني أنه لابد من تحقق جميع التعبيرات المطلوبة.
- **المعامل OR:** تعني (أو) يعني اختيار، تقوم بإضافة أكثر من تعبير لتحقيق الشرط، وعند استخدامها يكفي بتحقيق تعبير واحد من التعبيرات المطلوبة.

صيغة كتابة المعاملات المنطقية في الشروط:

```
if condition logical condition :
    statements
|
```

الشرح:

```

1 2 3 4 5
↓ ↓ ↓ ↓ ↓
File Edit Format Run Options Window Help
if condition logical condition :
    statement ← 6
| ← 7
```



١. تتم كتابة if في بداية السطر.
٢. تتم إضافة مسافة بعد if ثم كتابة الشرط.
٣. تتم إضافة مسافة ثم كتابة المعامل المنطقي and أو or.
٤. تتم إضافة مسافة بعد المعامل ثم كتابة الشرط.
٥. بعد الانتهاء من الشروط يجب وضع نقطتين رأسييتين ثم الضغط على مفتاح Enter.
٦. هنا يتم كتابة الأحداث الخاصة في الشرط if.
٧. تتم إعادة المؤشر لبداية السطر للخروج من الشرط.

مثال على المعامل and:

اكتب برنامج يقوم بطلب عمر المستخدم، حيث إن كان عمره بين الـ 20 و 30 يقوم بطباعة الجملة: عمرك مناسب، أما غير ذلك يقوم بطباعة الجملة: عمرك غير مناسب.

```
x=int(input("age: "))
if x>20 and x<30 :
    print("عمرك مناسب")
else:
    print("عمرك غير مناسب")
```

النتيجة بعد إدخال رقم أصغر من 30:

```
age: 29
عمرك مناسب
>>>
```

النتيجة بعد إدخال رقم أكبر من 30:

```
age: 35
عمرك غير مناسب
>>>
```

النتيجة بعد إدخال رقم أصغر من 20:

```
age: 18
عمرك غير مناسب
>>>
```

كما تلاحظ تم حصر العمر المناسب بين 20 و 30، فعند إدخال رقم أقل أو أكثر من ذلك ينفذ الشرط بطريقة دقيقة جداً.

مثال على المعامل or:

اكتب برنامج يقوم بطلب عمر المستخدم، حيث إن كان عمره بين الـ 25 و 30 أو 35 و 40 يقوم بطباعة الجملة (عمرك مناسب)، أما غير ذلك يقوم بطباعة الجملة (عمرك غير مناسب).



```
x=int(input("age: "))
if x>25 and x<30 or x>35 and x<40 :
    print("عمرک مناسب")
else:
    print("عمرک غير مناسب")
```

النتيجة بعد إدخال عمر 23:

```
age: 23
عمرک غير مناسب
>>>
```

النتيجة بعد إدخال عمر 26:

```
age: 26
عمرک مناسب
>>>
```

النتيجة بعد إدخال عمر 34:

```
age: 34
عمرک غير مناسب
>>>
```

النتيجة بعد إدخال عمر 38:

```
age: 38
عمرک مناسب
>>>
```

النتيجة بعد إدخال عمر 44:

```
age: 44
عمرک غير مناسب
>>>
```

كما تلاحظ تم تحديد فئتين عمريتين لتكون مناسبة، فعند محاولة إدخال عمر أكبر أو أصغر أو بين الفئتين تكون النتيجة غير مناسب، لكن عند إدخال العمر ضمن الفئتين يتم طباعة العمر مناسب.



تمارين الوحدة

١. ما هو المخطط الانسيابي؟
٢. عرف الجمل الشرطية مع ذكر أنواعها؟
٣. ارسم المخطط الانسيابي لدالة if و if-elif-else
٤. اكتب البرامج التالية مع رسم مخطط انسيابي لكل برنامج:
 - برنامج يطلب من المستخدم إدخال رقمين ومقارنة حاصل جمعهما بالرقم 15، بحيث إذا كان المجموع أكبر من الرقم 15 يطبع الرسالة (انت مؤهل للحصول على الجائزة).
 - برنامج يطلب من المستخدم إدخال عمره، عندما يكون عمر المستخدم أكبر من 20 يطبع الرسالة (تم قبورك في الرحلة)، أما إذا عمر المستخدم أصغر من 20 يطبع الرسالة (لم يتم قبورك في الرحلة).
 - برنامج يطلب من المستخدم إدخال رقم بين 1 و3، بحيث إذا كان الرقم 1 يطبع (يوم السبت)، أما إذا الرقم 2 يطبع (يوم الأحد)، أما إذا الرقم 3 يطبع (يوم الاثنين).
 - برنامج يطلب عمر المستخدم وطوله، بحيث إذا كان العمر أكبر من 35 والطول أطول من 170، يطبع العبارة (تتطبق الشروط)، وإذا كان العمر أصغر من 20 والطول أقصر من 150، يطبع عبارة (لا تتطبق الشروط)، وغير ذلك يطبع عبارة (في قائمة الانتظار).



نموذج تقييم المتدرب لمستوى أدائه				
يعبأ من قبل المتدرب نفسه وذلك بعد الانتهاء من تمارين الوحدة				
بعد الانتهاء من التدريب على وحدة الاختيار بالجمل الشرطية، قيم نفسك وقدراتك بواسطة إكمال هذا التقييم الذاتي بعد كل عنصر من العناصر المذكورة، وذلك بوضع علامة (✓) أمام مستوى الأداء الذي أتقنته، وفي حالة عدم قابلية المهمة للتطبيق ضع العلامة في الخانة الخاصة بذلك.				
م	العناصر	مستوى الأداء (هل أتقنت الأداء)		
		غير قابل للتطبيق	لا	جزئياً
١	يستخدم الجمل الشرطية if، if-else، if-elif-else في لغة بايثون.			
٢	يرسم المخططات الانسيابية للجمل الشرطية.			
٣	ينفذ الجمل الشرطية باستخدام دوال المقارنة.			
يجب أن تصل النتيجة لجميع المفردات (البند) المذكورة إلى درجة الإتقان الكلي أو غير قابلة للتطبيق، وفي حالة وجود مفردة في القائمة "لا" أو "جزئياً" فيجب إعادة التدريب على هذا النشاط مرة أخرى بمساعدة المدرب.				



نموذج تقييم المدرب لمستوى أداء المتدرب				
يعبأ من قبل المدرب وذلك بعد الانتهاء من تمارين الوحدة				
اسم المتدرب :		التاريخ:		
رقم المتدرب :		المحاولة: ١ ٢ ٣ ٤		
		العلامة:		
كل بند أو مفردة يقيم بـ ١٠ نقاط الحد الأدنى: ما يعادل ٨٠٪ من مجموع النقاط. الحد الأعلى: ما يعادل ١٠٠٪ من مجموع النقاط.				
م	بنود التقييم	النقاط (حسب رقم المحاولات)		
		١	٢	٣
١	يستخدم الجمل الشرطية if ، if-else ، if-elif-else في لغة بايثون.			
٢	يرسم المخططات الانسيابية للجمل الشرطية.			
٣	ينفذ الجمل الشرطية باستخدام دوال المقارنة.			
المجموع				
ملحوظات:				
توقيع المدرب:				



المراجع

م	المراجع
١	Introduction to Programming Using Python -Y. Daniel Liang
٢	Lean Python Learn Just Enough Python to Build Useful Tools - Paul Gerrard
٣	تعلم البرمجة مع بايثون ٣ – ترجمة: هشام رزق الله
٤	فكر بايثون كيف تفكر كعالم حاسوب - ألن داووني
٥	https://www.w3schools.com/python/default.asp
٦	https://www.programiz.com/python-programming
٧	https://www.youtube.com/playlist?list=PLMYF6NkLrdN98I0nEXOuR_gK8b4w-NJcN
٨	https://www.youtube.com/playlist?list=PLF8OvnCBIEY1j4hxoqXqJk08ASU7D_W87
٩	حقيبة برمجة الحاسب الآلي (دعم فني)